

Edge Computing with AWS IoT Greengrass - *Part #1*

by Cagatay Sonmez, 30 Oct 2026

The Primary Sources Used in This Lecture

- https://docs.aws.amazon.com/greengrass/v2/developerguide
- https://docs.aws.amazon.com/greengrass/v2/developerguide/setting-up.html
- https://docs.aws.amazon.com/greengrass/v2/developerguide/develop-greengrass-components.html
- https://docs.aws.amazon.com/greengrass/v2/developerguide/ip-detector-component.html
- https://docs.aws.amazon.com/greengrass/v2/developerguide/interprocess-communication.html

- All the source codes used in this Lecture can be found in the following GitHub repository:
  - https://github.com/CagataySonmez/AWS-IoT-Greengrass-Demo

Prerequisite: Amazon Web Services (AWS)

- **What is AWS?**
  - It is a cloud platform built on the "**XaaS**" (Everything as a Service) philosophy.
  - Think of AWS as a massive, **on-demand toolbox** of digital services—like storage, servers, and security—that you can access over the internet.
  - We **don't need** to buy or build our own data centers.
- **Why Are We Using It Today?**
  - AWS acts as the cloud control plane for deploying, managing, and monitoring our edge applications. The applications themselves execute locally on the edge device.
  - We use it to:
    - **Deploy** our code.
    - **Manage** our device from anywhere.
    - **Communicate** with our device securely .

Our Core Services in this Lecture (Identity & Storage)



IAM (Identity & Access Management)

- **What it is:** The "Security Guard" for your AWS account. It manages *who* (Users, Devices) can access what (Services).
- **How we use it:** To create *Roles* (an "identity" for our device) and *Policies* (a "permission slip" in JSON format) that allow our device to securely talk to the cloud .



S3 (Simple Storage Service)

- **What it is:** A giant "Hard Drive in the Cloud." You store your files in folders called "*Buckets*".
- **How we use it:** We store our Python code (called *Artifacts*) in an S3 Bucket . Greengrass then automatically downloads this code from S3 to the device .

Our Core Services in this Lecture (Compute & Messaging)



AWS Lambda

- **What it is:** A "serverless" service. It lets you run code without thinking about servers. You just upload your script, and AWS runs it for you.
- **How we use it:** We'll create a Lambda function in the cloud and then show how Greengrass can *import* it and run that same function locally on our edge device .



AWS IoT Core

- **What it is:** The "Post Office" or "Message Hub" for all your IoT devices. It's a secure *MQTT Broker* that handles millions of messages.
- **How we use it:** To send messages *from* the cloud to our device (e.g., 'cmpe/request') and receive responses back . We also use its *MQTT Test Client* to check our work .

Key Concept to Remember:

- **Region**: A geographic location containing multiple isolated Availability Zones.
- Greengrass resources (S3 Bucket, IoT Thing, Lambda) must be in the same region!

⚠ Reminder

Monitor AWS **Free Tier** usage via **Budget Alerts** to avoid unexpected charges!

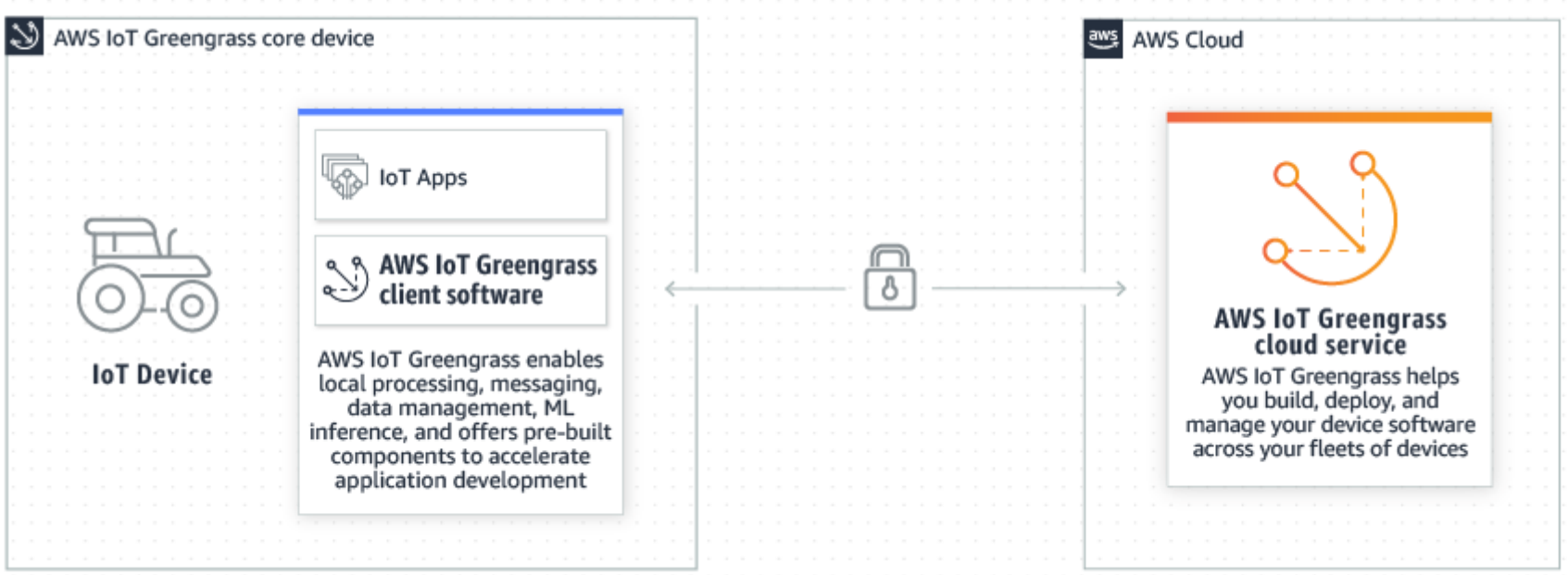
- ✓ **Go to AWS Billing Dashboard** – You can find this under your account settings.
- ✓ **Create a Budget** – Set a budget for $0 or a minimal amount to receive alerts as soon as usage might incur charges.
- ✓ **Configure Alerts** – Enable notifications so you'll get an email if your usage approaches your set limit.

⚠ Budget alerts are not real-time spending limits. Charges may occur before an alert is received.



What is AWS IoT Greengrass?

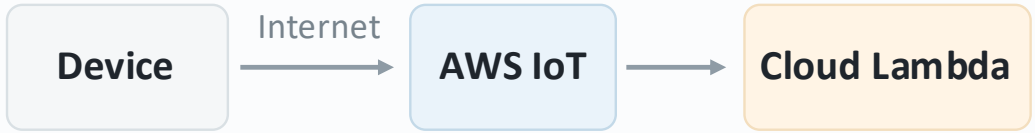
- AWS IoT Greengrass is software that extends cloud capabilities to local devices. This enables devices to collect and analyze data closer to the source of information, react autonomously to local events, and communicate securely with each other on local networks. Local devices can also communicate securely with AWS IoT Core and export IoT data to the AWS Cloud[1].



[1] https://docs.aws.amazon.com/greengrass/v2/developerguide/what-is-iot-greengrass.html

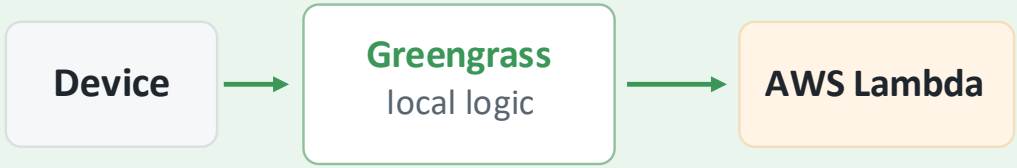
Why Greengrass Instead of Cloud-Only Lambda?

CLOUD-ONLY ARCHITECTURE



All data traverses network before processing.

EDGE + CLOUD ARCHITECTURE



Local decisions; selected data sent to cloud.

LOWER LATENCY

React immediately to sensors, controls, and events.

OFFLINE RESILIENCE

Keep local logic running when connectivity drops.

LESS DATA TRANSFER

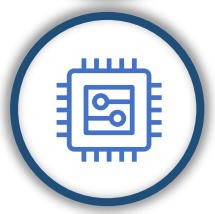
Filter, aggregate, or summarize before upload.

Use both: Greengrass for local processing + cloud Lambda for scalable back-end workflows.

AWS IoT Greengrass Key Concepts



Greengrass Core Device: A device that runs the AWS IoT Greengrass Core software. A Greengrass core device is an AWS IoT thing. You can add multiple core devices to AWS IoT thing groups. This could be a Raspberry Pi, an industrial PC, or any other compatible device.



Greengrass Client Device: Any device (such as an IoT device, mobile app, or system) that communicates with a Greengrass Core device locally or via AWS IoT Core over MQTT, using the AWS IoT Device SDK.



Greengrass Core (Client) Software: The set of all AWS IoT Greengrass software that you install on a core device. AWS IoT Greengrass Core software comprises the Nucleus and other components. It is provided by AWS that enables a device to connect to AWS IoT Core.



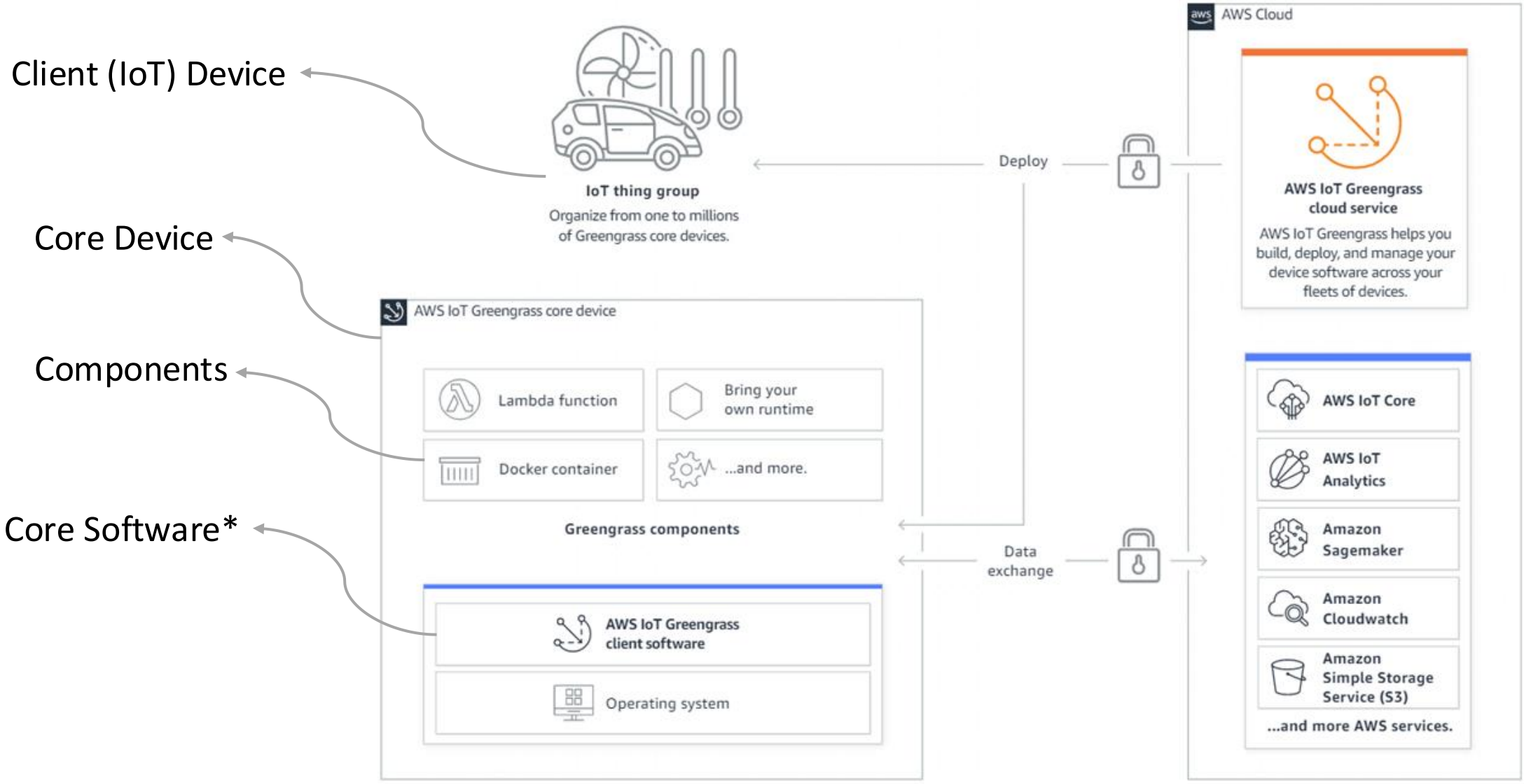
Greengrass Component: A software module (e.g. Python script) that is deployed to and runs on a Greengrass core device. These modules are modeled as components which can perform various tasks, such as collecting data from sensors, controlling actuators, or running machine learning models.



Deployment: The process of installing and configuring Greengrass components on a Greengrass Core device. This typically involves uploading the component code and configuration files to the device.

Physical Device Software Concept Action/Process

How AWS IoT Greengrass works



*Greengrass *core software* is also called Greengrass *client software* in AWS documentation. Client devices use the IoT Device SDK

Selecting Client's Operating System

- Some features are supported on only certain operating systems!

Security

Feature	Linux	Windows
Use a hardware security module (HSM) to securely store the device's private key and certificate	Yes	No

Remote maintenance and updates

Feature	Linux	Windows
Manage core devices with AWS Systems Manager	Yes	No
Connect to core devices with AWS	Yes	No

Component features

Feature	Linux	Windows
Deploy and invoke Lambda functions	Yes	No
Publish messages to Amazon Simple Notification Service using the Amazon SNS component	Yes	No
Amazon Kinesis Data Firehose delivery streams using the Kinesis Data Firehose component	Yes	No
Video Streams using the edge connector for Kinesis Video Streams component	Yes	

Installation

Feature	Linux	Windows
Run AWS IoT Greengrass in a Docker container using a prebuilt Docker image	Yes	No

Machine learning

Feature	Linux	Windows
Perform machine learning inference using Amazon Lookout for Vision	Yes	No

Full feature list: https://docs.aws.amazon.com/greengrass/v2/developerguide/operating-system-feature-support-matrix.html

AWS IoT Greengrass Core Software

AWS IoT Greengrass Core Software

- The AWS IoT Greengrass Core software extends AWS functionality onto an AWS IoT Greengrass core device
- The AWS IoT Greengrass Core software provides a lot of functionality:
  - Deploy and invoke components and Lambda functions.
  - Support MQTT messaging between AWS IoT and components.
  - Interact with local devices that connect and communicate over MQTT.
  - Execution of ML models on edge devices, enabling low-latency, real-time analytics.
  - Efficient and reliable high-volume IoT data transfer to the AWS Cloud using Stream Manager.
  - Perform secure, over-the-air (OTA) software updates.
  - Provide secure, encrypted storage of local secrets.
  - And more…

Steps for Greengrass Core SW Installation

1. Set up your environment and install the required tools
2. Create temporary AWS credentials (using a temporary user)
3. Set up a Greengrass Core device in the AWS Management Console
4. Install the Greengrass Core sw using the temporary user's credentials
5. Remove the temporary user after installation

Step 1: Set Up Your Environment

For Linux Based Devices

- AWS IoT Greengrass Core software requires Java version 8 or higher.
- Amazon recommends to use OpenJDK 11 or Amazon Corretto 11.

For Debian-based or Ubuntu-based distributions:

```
sudo apt install default-jdk
```

For Red Hat-based distributions:

```
sudo yum install java-11-openjdk-devel
```

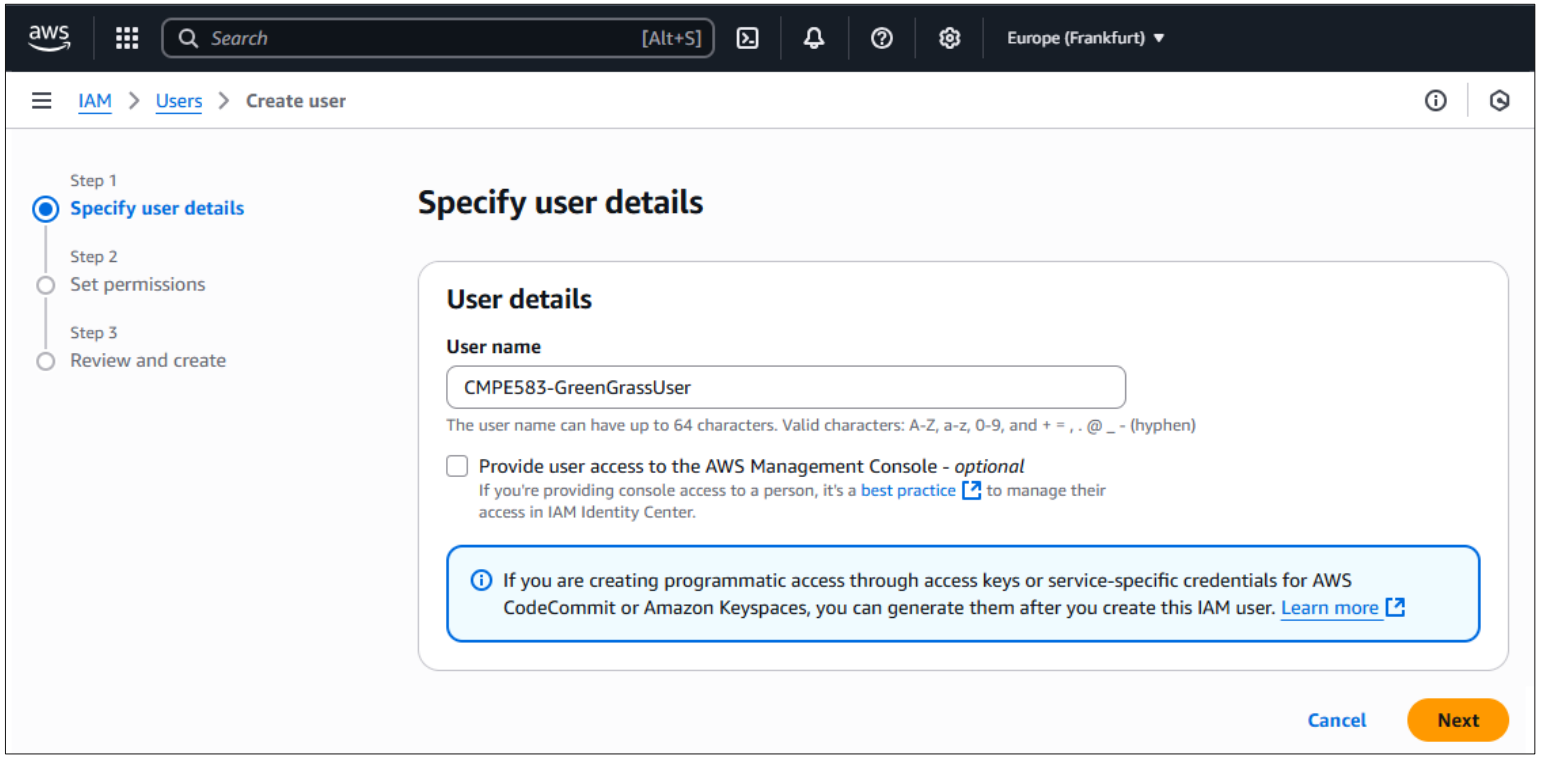
For Amazon Linux 2023 (AL2023):

```
sudo dnf install java-11-amazon-corretto -y
```

Step 2: Create AWS Credentials

Create a Temporary User

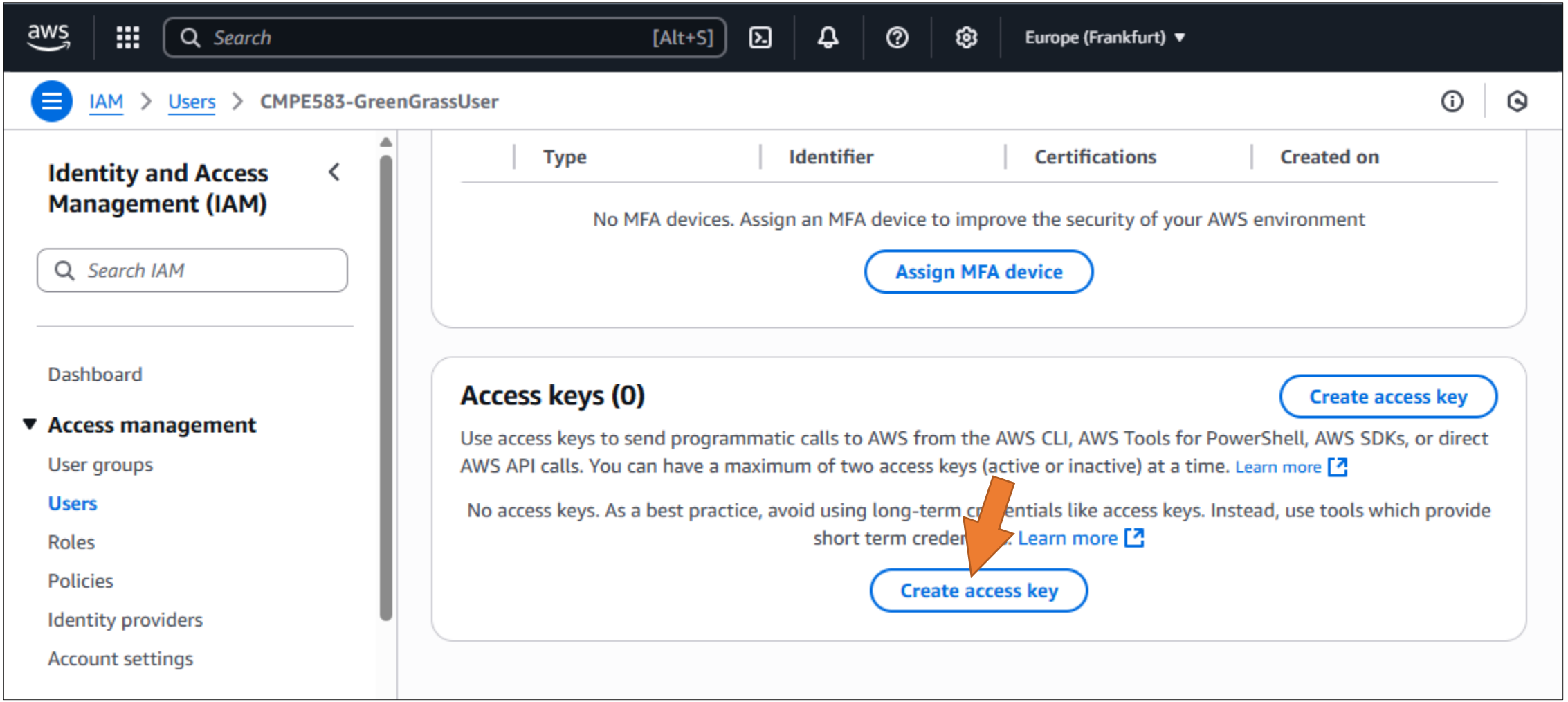
- Greengrass core software installer uses the security credentials to make programmatic requests for AWS resources.
- We use a temporary user to install core software.
- We will remove this user after the installation is completed.



★ A temporary IAM user and access key are used for this classroom demo; in production, prefer short-lived credentials instead of long-lived access keys.

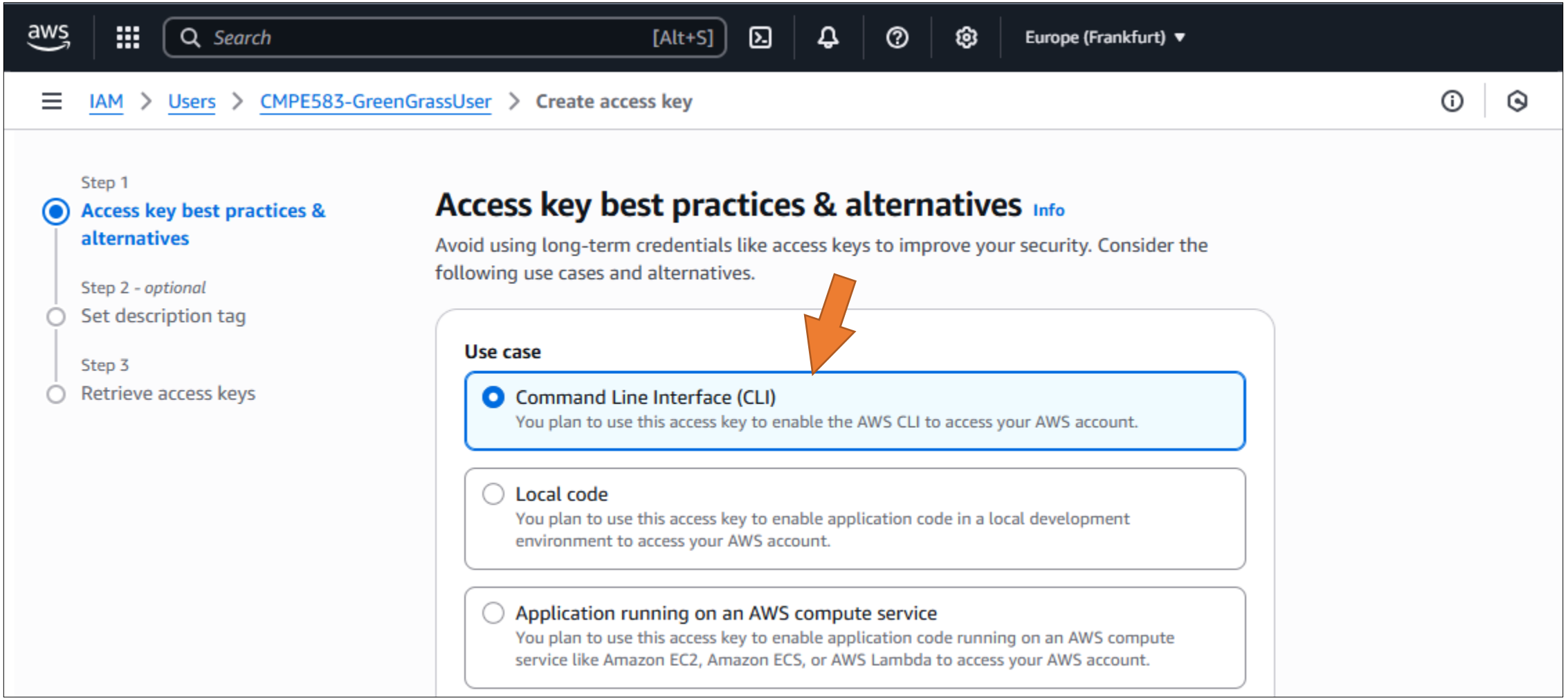
Step 2: Create AWS Credentials

Create Access Keys for the User



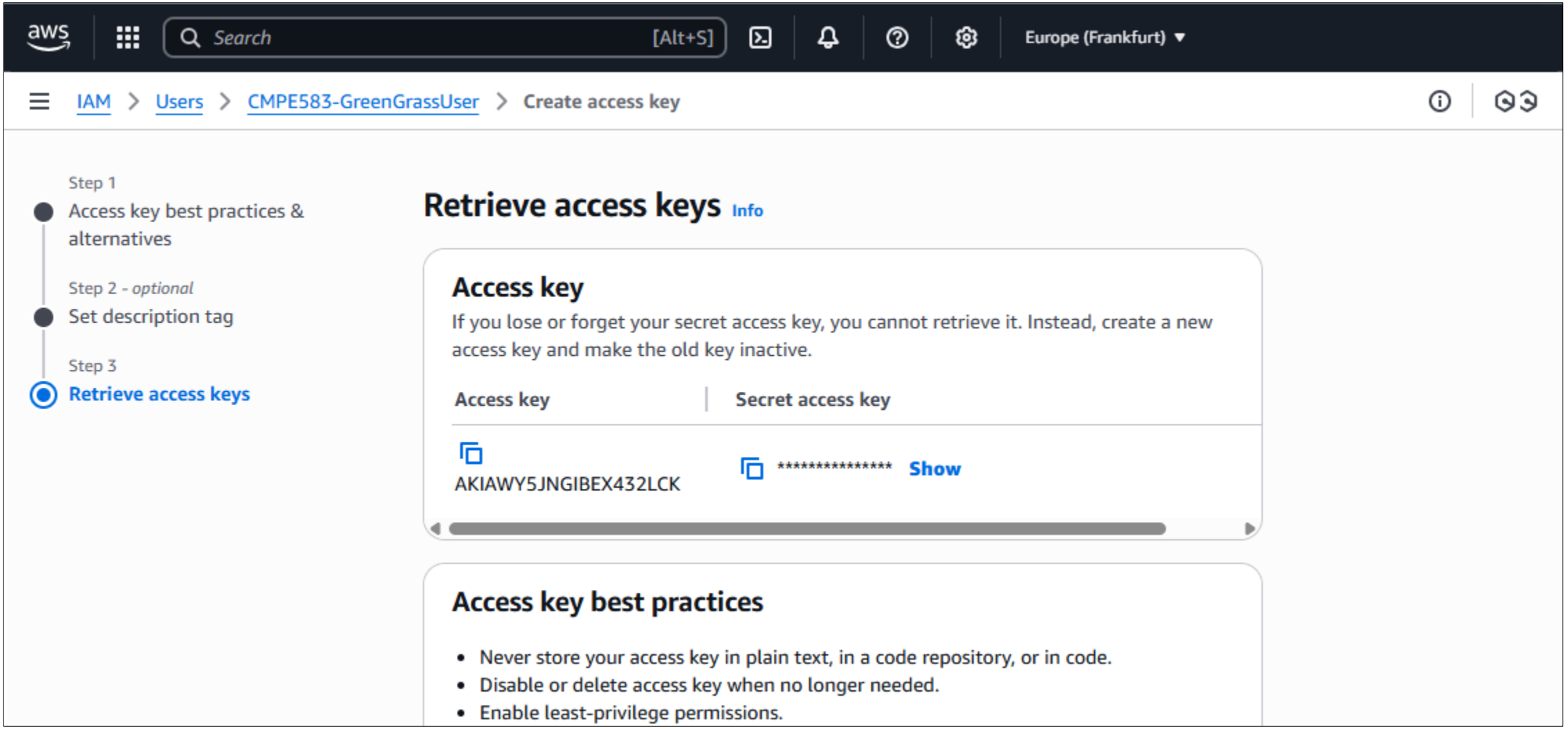
Step 2: Create AWS Credentials

Define Access Key Use-Case

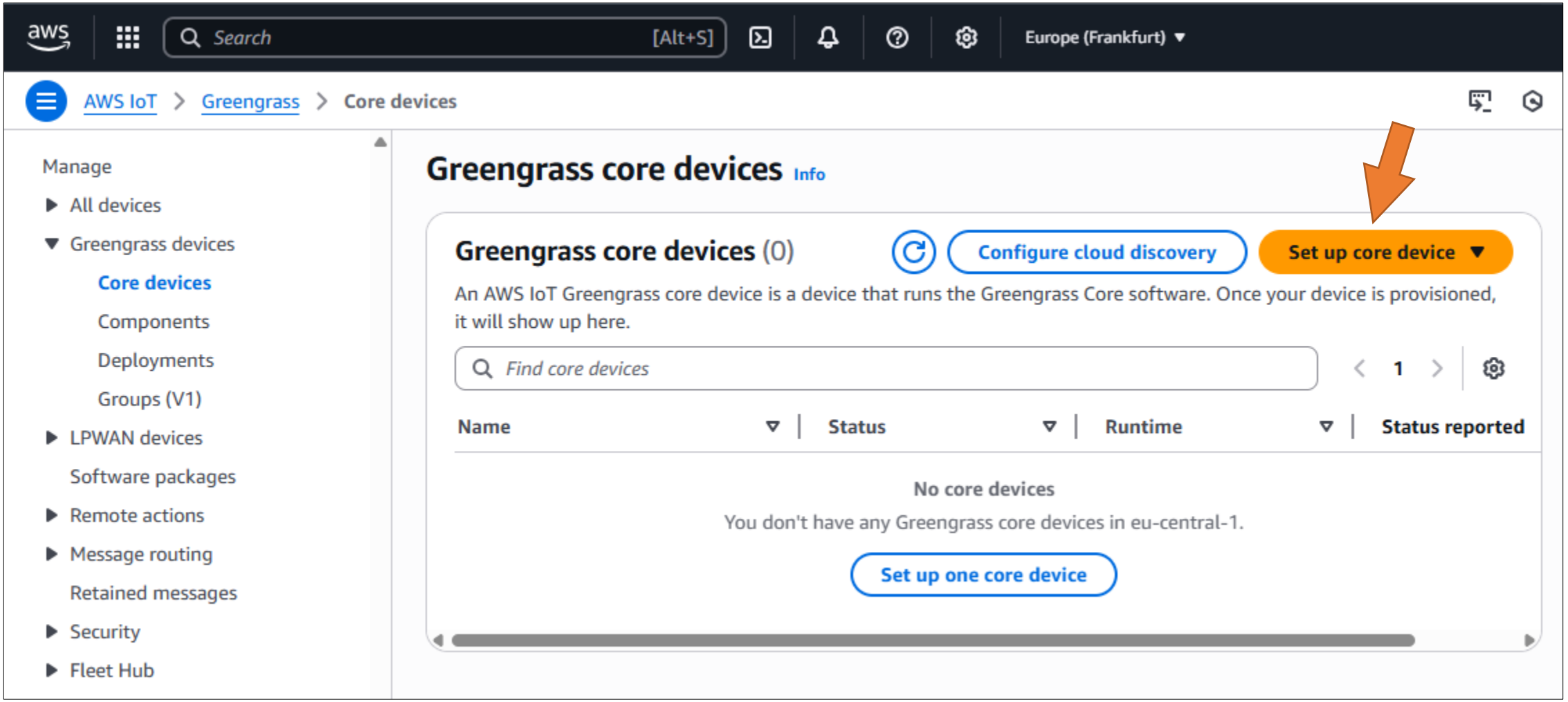


Step 2: Create AWS Credentials

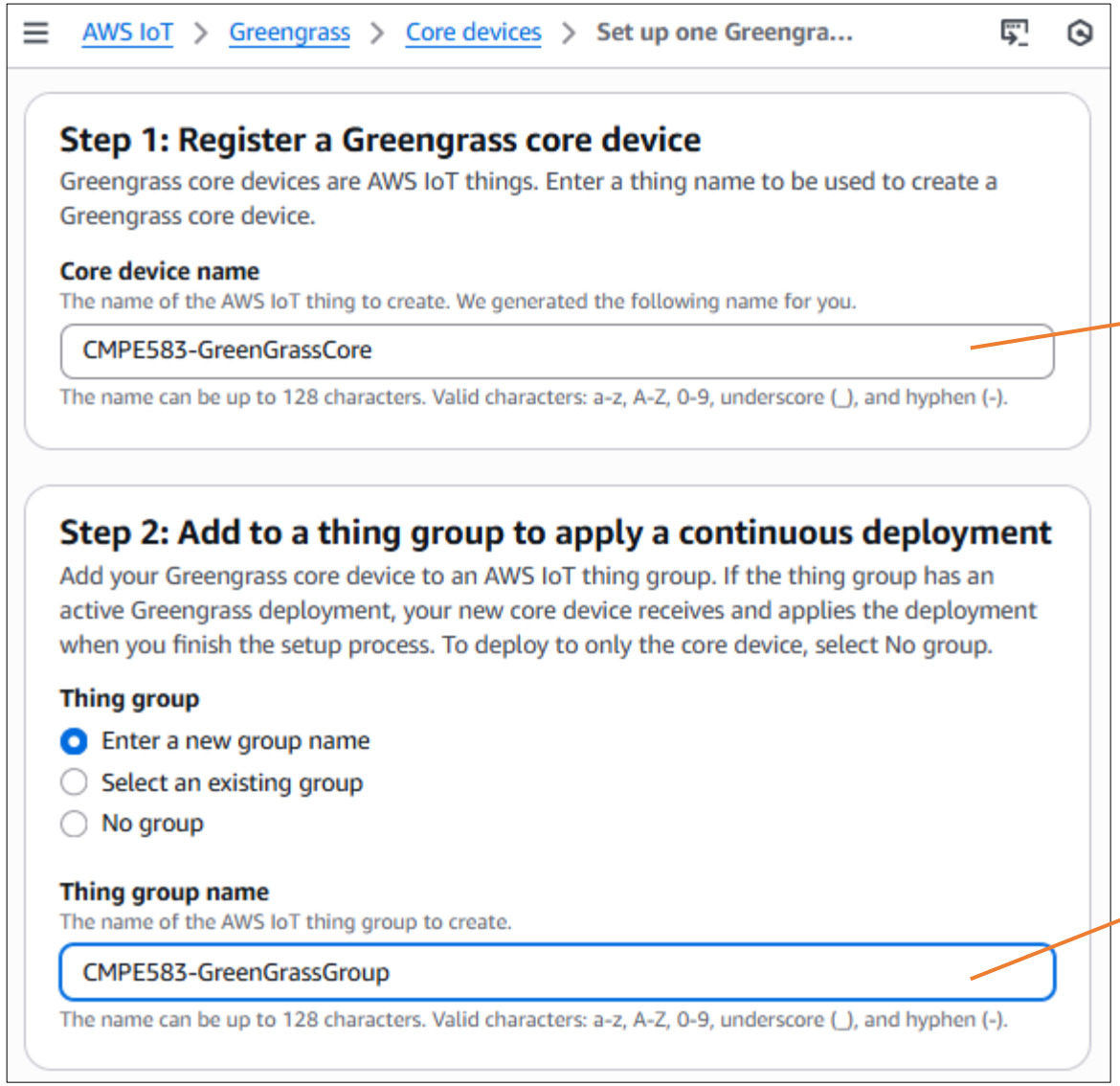
Copy the Access Keys



Step 3: Set Up a Greengrass Core Device in AWS



Step 3: Set Up a Greengrass Core Device in AWS

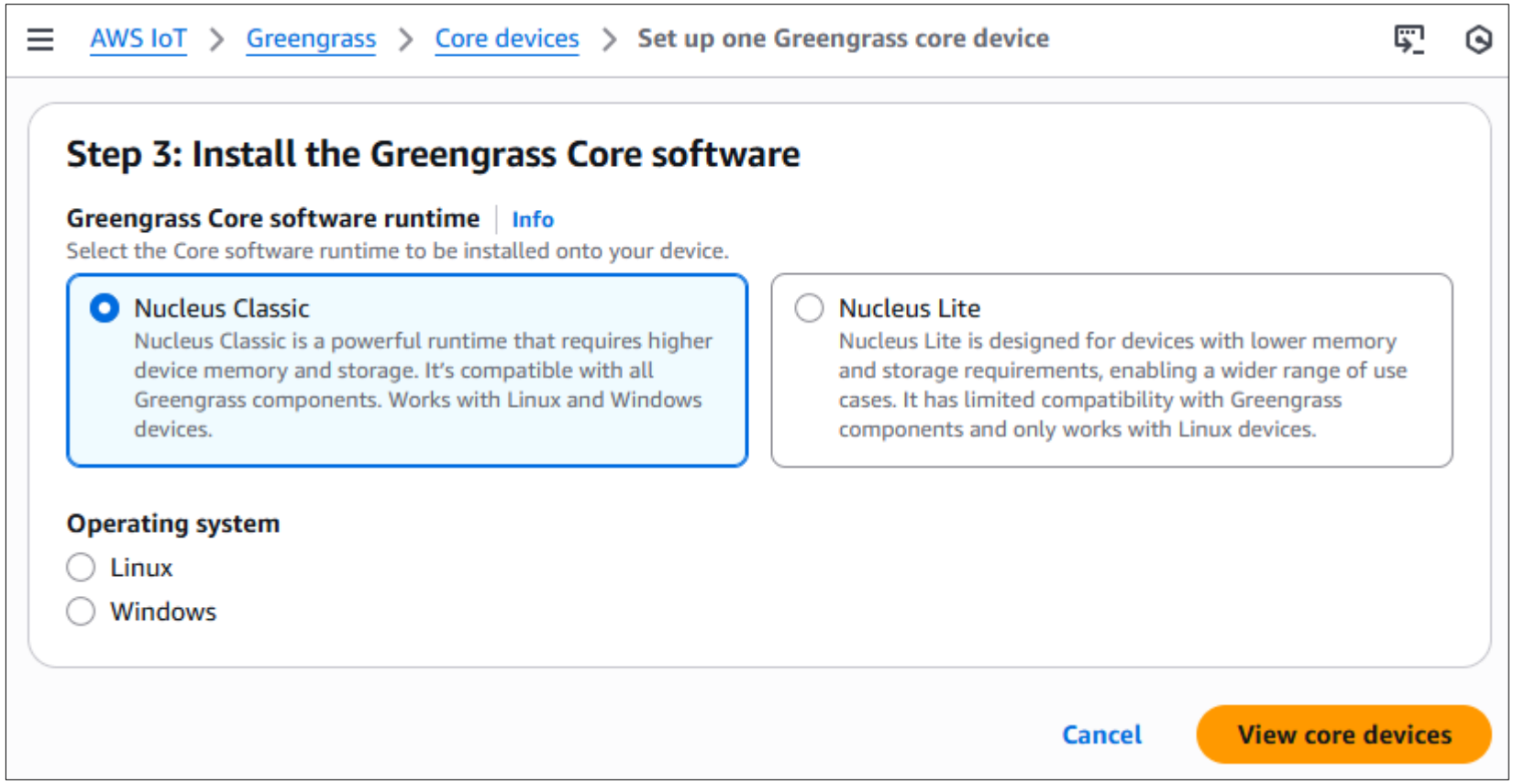


The name of the AWS IoT thing for your Greengrass core device.

The name of the new group to create to apply a continuous deployment.

Step 3: Set Up a Greengrass Core Device in AWS

Select Your Runtime



Step 3: Set Up a Greengrass Core Device in AWS

Greengrass Nucleus vs. Nucleus Lite

FEATURE	NUCLEUS	NUCLEUS LITE
Best suited for	Gateways, industrial PCs, richer edge platforms	Resource-constrained embedded Linux devices
Implementation	JVM-based runtime (Java required)	Lightweight runtime; no JVM required
OS support	Linux and Windows	Linux with systemd
Component types	Generic + Lambda components	Generic components; no Lambda components
Messaging	Cloud MQTT, local Pub/Sub, client-device MQTT	Cloud MQTT + local Pub/Sub; no client-device MQTT
Resource profile	Typically more resources (>128 MB RAM guidance)	Small runtime (~15 MB RAM; 5 MB storage)

For this lecture: Nucleus (Classic) Supports our Python components and leaves room for Lambda-based demos.

Step 4: Install the Greengrass Core Software

Configure AWS Credentials on the Client

- The Greengrass installer uses AWS credentials to provision the AWS resources that it requires.

  Provide credentials as environment variables before running the installer:

```
export AWS_ACCESS_KEY_ID=<AWS_ACCESS_KEY_ID>
export AWS_SECRET_ACCESS_KEY=<AWS_SECRET_ACCESS_KEY>
export AWS_SESSION_TOKEN=<AWS_SESSION_TOKEN>
```

- AWS_SESSION_TOKEN is only required for temporary session credentials
- Credentials are not saved by the Greengrass installer!

Step 4: Install the Greengrass Core Software

For Linux Based Devices

- AWS IoT provides an installer that you can use to set up a Greengrass core device.
- The installer provisions the Greengrass core device as an AWS IoT thing and connects the device to AWS IoT.

Download the installer:

```
curl -s https://d2s8p88vqu9w66.cloudfront.net/releases/greengrass-nucleus-latest.zip > greengrass-
    nucleus-latest.zip && unzip greengrass-nucleus-latest.zip -d GreengrassInstaller
```

Run the installer:

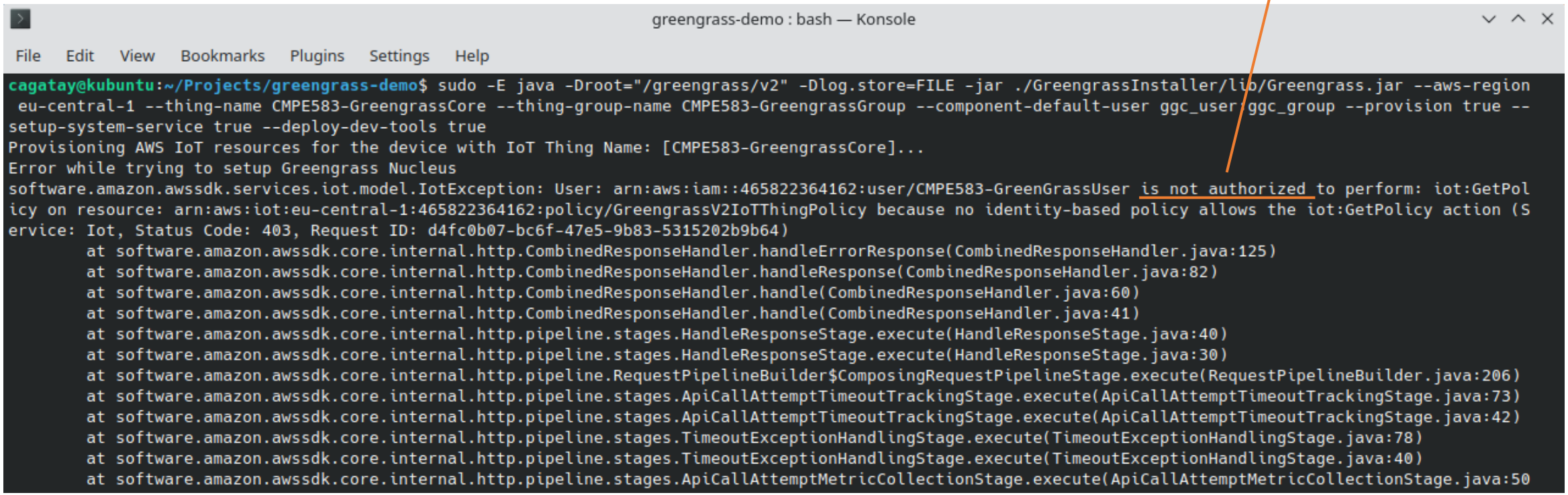
```
sudo -E java -Droot="/greengrass/v2" -Dlog.store=FILE -jar ./GreengrassInstaller/lib/Greengrass.jar --aws-
    region eu-central-1 --thing-name CMPE583-GreenGrassCore --thing-group-name CMPE583-
    GreenGrassGroup --component-default-user ggc_user:ggc_group --provision true --setup-system-service
    true --deploy-dev-tools true
```

★Up-to-date installer commands are provided in AWS Console while creating a core device!

Step 4: Install the Greengrass Core Software

Run the Installer Command

The user is not authorized to perform installer jobs! You should attach required permissions!

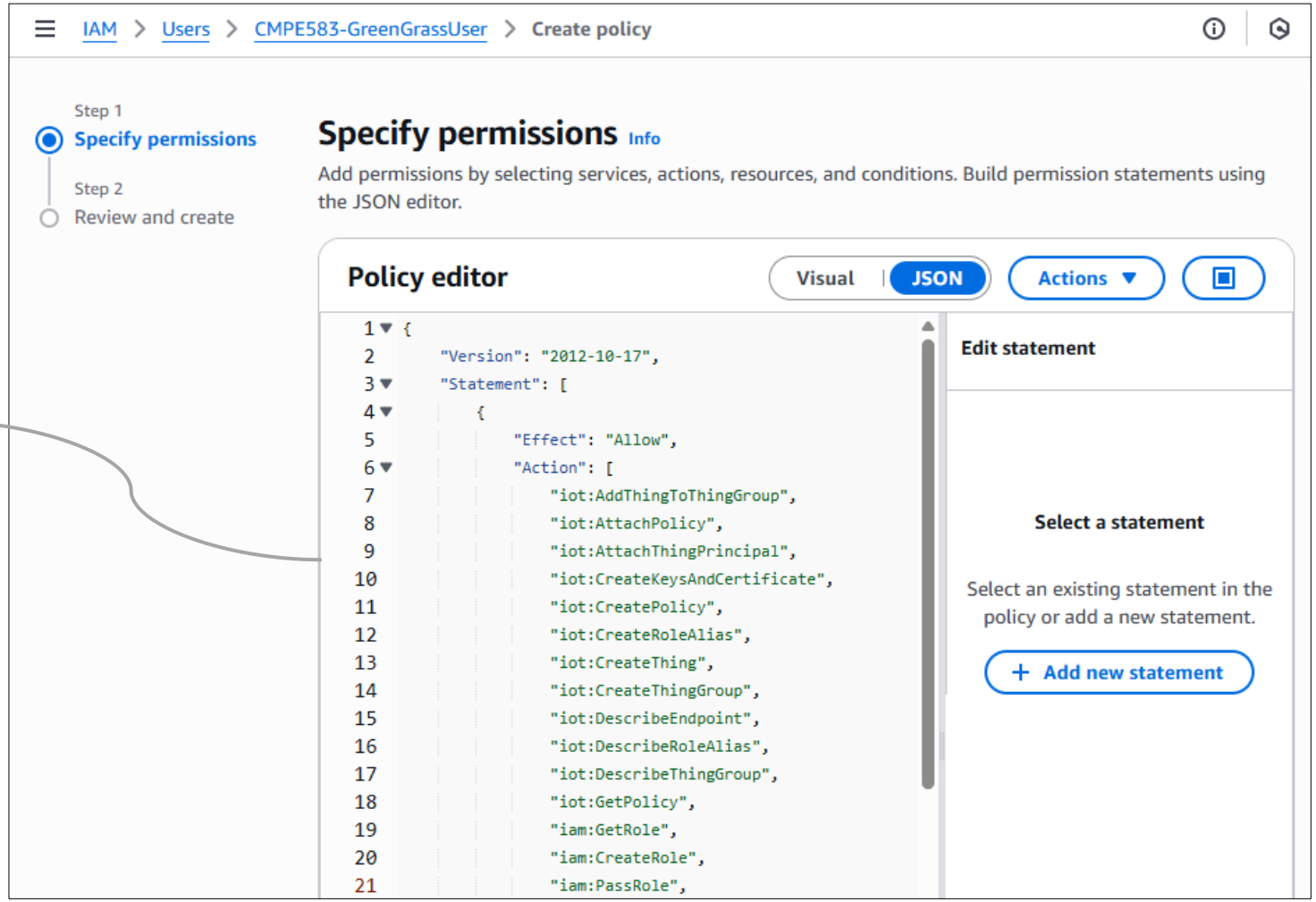


Step 4: Install the Greengrass Core Software

Set Required Permissions to Greengrass User

- When installing a core device, the user we created needs certain permissions to provision AWS IoT resources.

Permission content available in GitHub
- *AWS/IAM/cmpe583-GreengrassUser-ProvisionPolicy.json*



Step 4: Install the Greengrass Core Software

Run the Installer Command Again

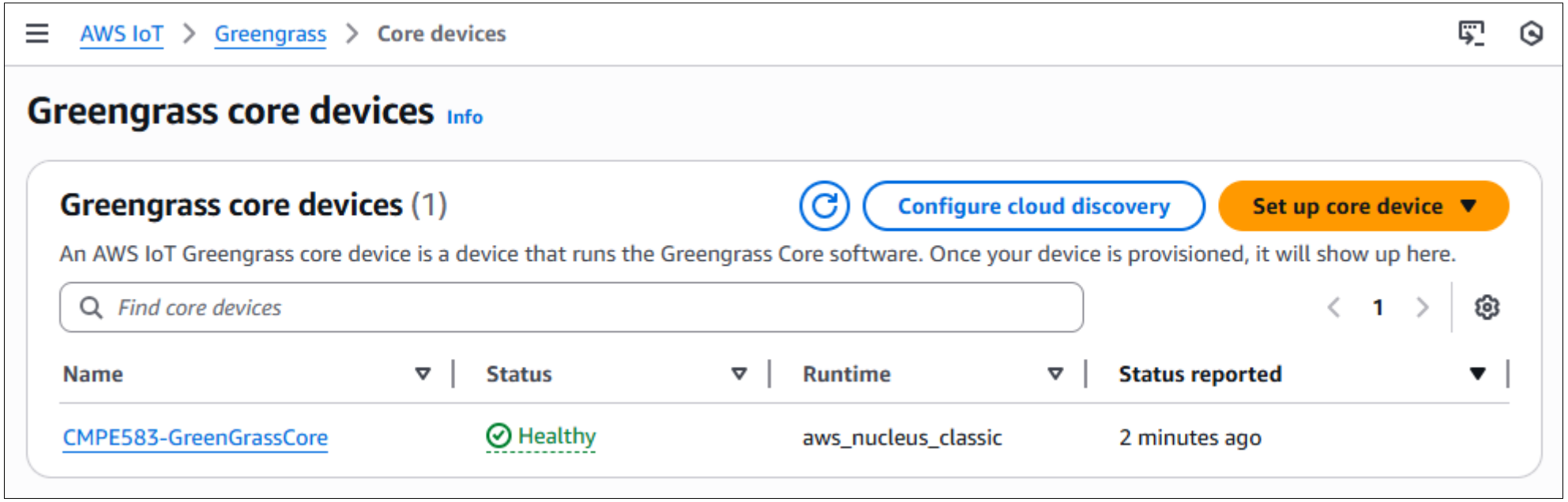
Notice: On Ubuntu 25.10 and later, the default sudo does not support preserving environment variables required by AWS IoT Greengrass. To fix this, switch to sudo.ws by running:

#sudo update-alternatives --set sudo /usr/bin/sudo.ws

```
Provisioning AWS IoT resources for the device with IoT Thing Name: [CMPE583-GreenGrassCore]...
Creating new IoT policy "GreengrassV2IoTThingPolicy"
Creating keys and certificate...
Attaching policy to certificate...
Creating IoT Thing "CMPE583-GreenGrassCore"...
Attaching certificate to IoT thing...
Successfully provisioned AWS IoT resources for the device with IoT Thing Name: [CMPE583-GreenGrassCore]!
Adding IoT Thing [CMPE583-GreenGrassCore] into Thing Group: [CMPE583-GreenGrassGroup]...
Successfully added Thing into Thing Group: [CMPE583-GreenGrassGroup]
Setting up resources for aws.greengrass.TokenExchangeService ...
TES role alias "GreengrassV2TokenExchangeRoleAlias" does not exist, creating new alias...
TES role "GreengrassV2TokenExchangeRole" does not exist, creating role...
IoT role policy "GreengrassTESCertificatePolicyGreengrassV2TokenExchangeRoleAlias" for TES Role alias not exist, creating policy...
Attaching TES role policy to IoT thing...
No managed IAM policy found, looking for user defined policy...
No IAM policy found, will attempt creating one...
IAM role policy for TES "GreengrassV2TokenExchangeRoleAccess" created. This policy DOES NOT have S3 access, please modify it with your private
components' artifact buckets/objects as needed when you create and deploy private components
Attaching IAM role policy for TES to IAM role for TES...
Configuring Nucleus with provisioned resource details...
Downloading Amazon Root CAs
Created device configuration
Successfully configured Nucleus with provisioned resource details!
Creating a deployment for Greengrass first party components to the thing group
Configured Nucleus to deploy aws.greengrass.Cli component
Successfully set up Nucleus as a system service
```

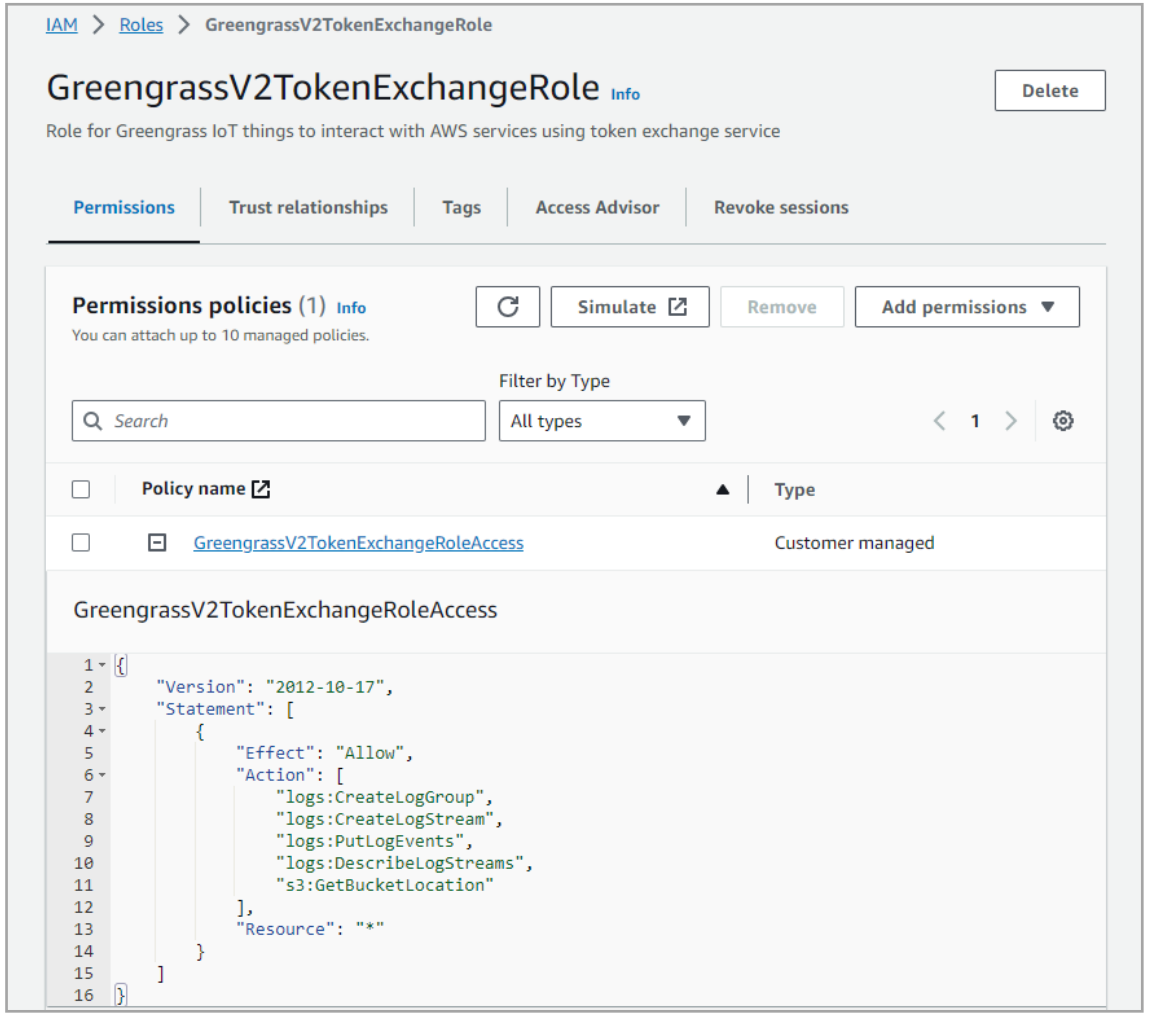
Step 4: Install the Greengrass Core Software

Rerun the Installer and Check If Core Device Is Healthy



Step 4: Install the Greengrass Core Software

Check If GreengrassV2TokenExchangeRole Is Created



- A role is created when you installed the AWS IoT Greengrass Core software.
- If you did not specify a name, the default is role name is *GreengrassV2TokenExchangeRole*.
- This role should have a policy named *GreengrassV2TokenExchangeRoleAccess*.
- If you want to use different Role and Policy names, check Installer help command.

★ You can delete the user that has been created after verifying the installation is successful!

Step 4: Install the Greengrass Core software

Troubleshooting

- If the GreengrassInstaller cannot create the GreengrassV2TokenExchangeRole and prints an error message, you can manually create a custom role in IAM console with the following JSON value and reinstall the core software:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "Service": "credentials.iot.amazonaws.com"
      },
      "Action": "sts:AssumeRole"
    }
  ]
}
```

Step 4: Install the Greengrass Core software

Monitor AWS IoT Greengrass logs

- The AWS IoT Greengrass Core software stores logs in the /greengrass/v2/logs folder on a core device

```
/greengrass/v2
└── logs
    ├── greengrass.log
    ├── greengrass_2021_09_14_15_0.log
    ├── ComponentName.log
    ├── ComponentName_2021_09_14_15_0.log
    └── main.log
```

AWS IoT Greengrass root folder

The AWS IoT Greengrass Core software rotates log files every hour or when they exceed a file size limit

- You must have root permissions to read AWS IoT Greengrass logs on the file system.

```
sudo tail -f /greengrass/v2/logs/greengrass.log
```

Step 4: Install the Greengrass Core Software

Auto Start of Greengrass Core Software

- If you installed the software as a system service, the installer runs the software at each startup.
- If you don't see following line in the installer output, Greengrass software will not start after rebooting the core device:

```
Successfully set up Nucleus as a system service
```

- In this case, you should run the software with the command below:

```
sudo /greengrass/v2/alts/current/distro/bin/loader
```

Uninstall the AWS IoT Greengrass Core Software

For Linux Based Devices

- If the software runs as a system service, you must stop, disable, and remove it.

```
sudo systemctl stop greengrass.service
sudo systemctl disable greengrass.service
sudo rm /etc/systemd/system/greengrass.service
```

- Verify that the service is deleted.

```
sudo systemctl daemon-reload && sudo systemctl reset-failed
```

- Remove the root folder from the device.

```
sudo rm -rf /greengrass/v2
```

Greengrass Components

Greengrass Components

- A component is a software module that runs on AWS IoT Greengrass core devices.
- Every component is defined by a recipe and may include one or more artifacts.

Recipe File

- Every component contains a recipe file.
- Recipes can be defined in JSON or YAML format.
- It defines component's metadata.
  - Configuration parameters
  - Component dependencies
  - Lifecycle
  - Platform compatibility

Artifacts

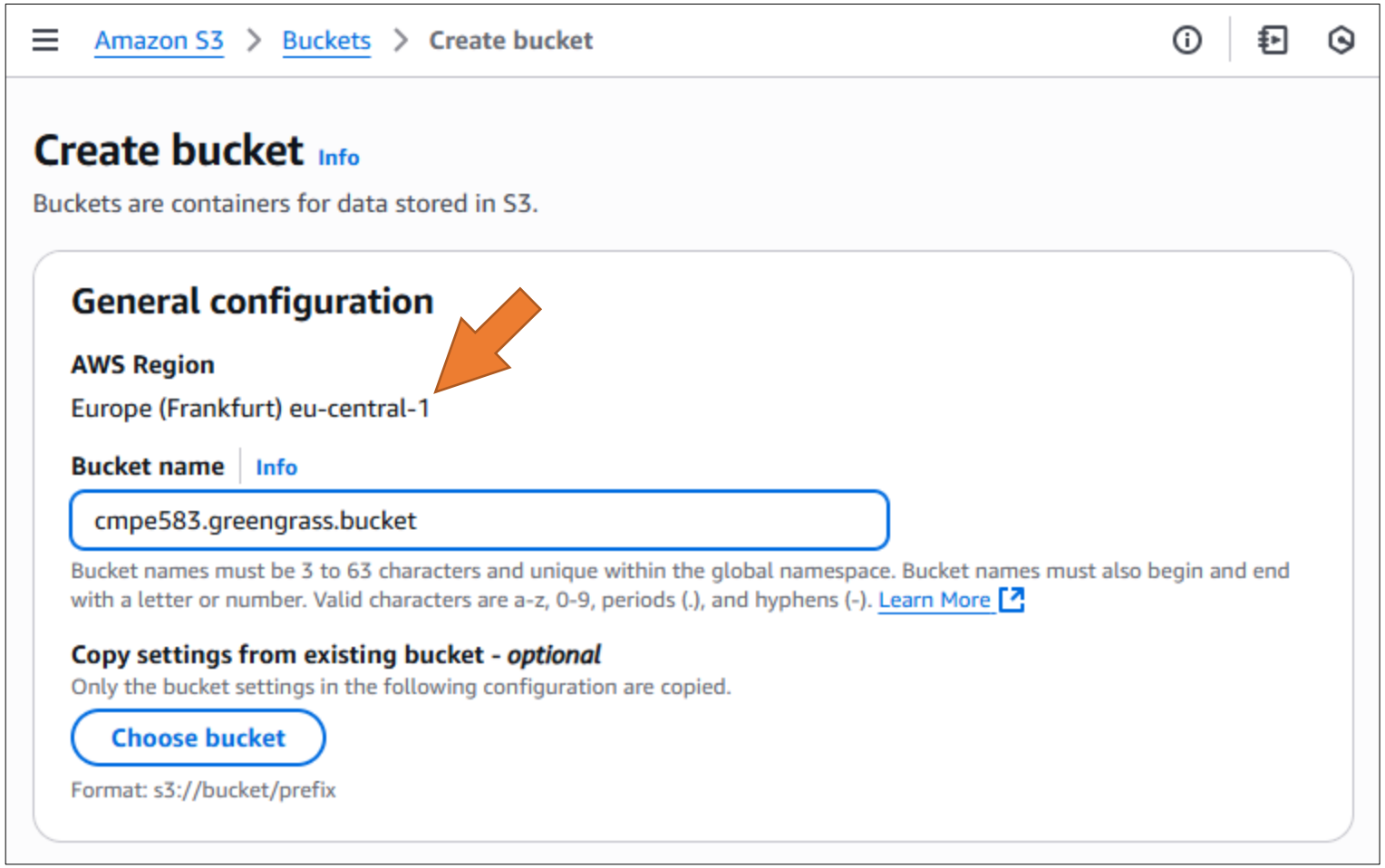
- Artifacts are component binaries.
- Components can have any number of artifacts.
- Artifacts can include:
  - Scripts
  - Compiled code
  - Static resources
  - Other files that a component consumes

Steps for Greengrass Component Deployment

1. Upload artifacts to Amazon S3
2. Set S3 access policies
3. Create the component and configure its recipe
4. Deploy the component via the AWS Management Console
5. Verify deployment in the console and on the device terminal

Step 1: Upload the Artifacts

Create a bucket via AWS S3 (Console)

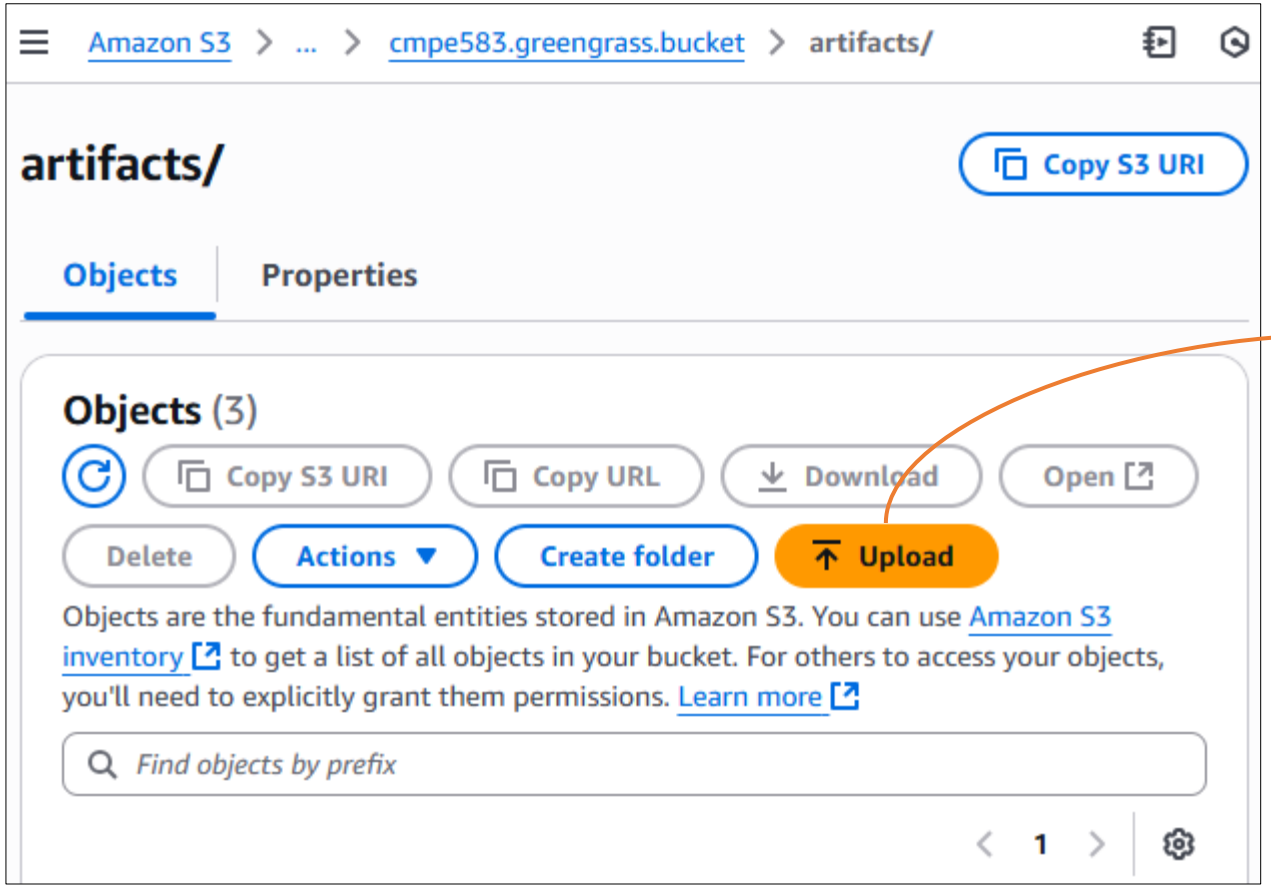


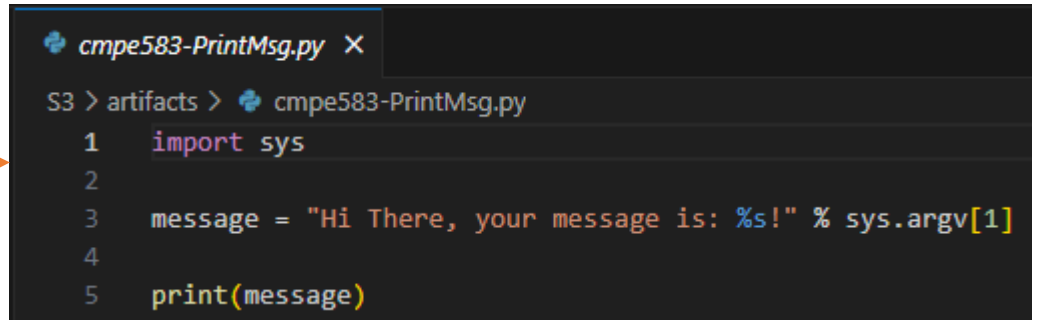
- Be sure that the S3 bucket is in the same AWS Region where you create the component.
- AWS IoT Greengrass doesn't support cross-Region requests for component artifacts!

Step 1: Upload the Artifacts

Create artifacts folder & upload files

- Upload your artifacts (source codes) to a folder in your S3 bucket.





Step 2: Set Policies to Access Files in S3

Via AWS IAM (Console)

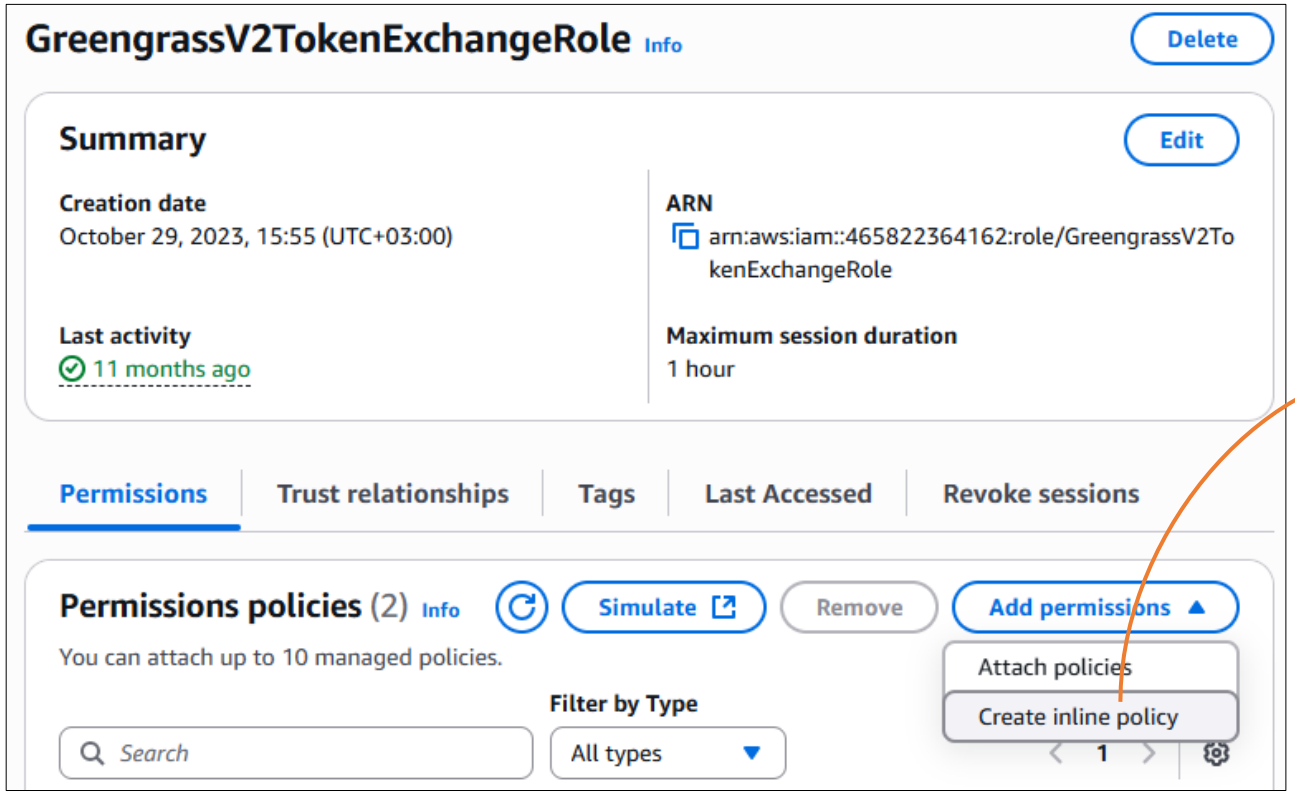
- Greengrass component artifacts should be stored in AWS S3 bucket.
- Therefore, you should allow the core device to access component artifacts in the S3 bucket.
  - Attach a policy similar to policy below to GreengrassV2TokenExchangeRole from AWS IAM console:

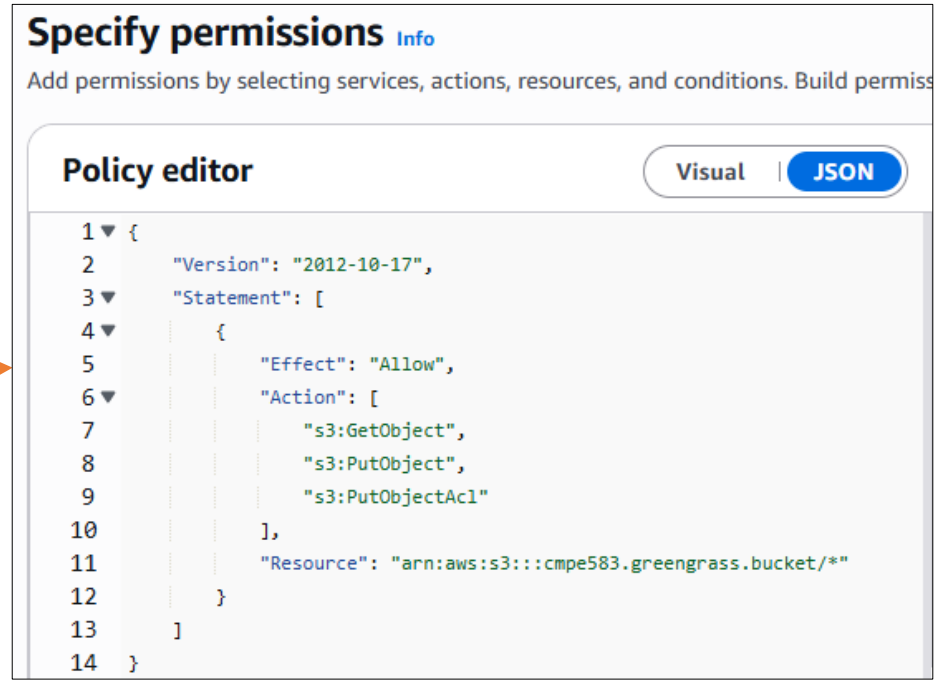
```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Effect": "Allow",
            "Action": [
                "s3:GetObject"
            ],
            "Resource": "arn:aws:s3:::cmpe583.greengrass.bucket/*"
        }
    ]
}
```

Step 2: Set Policies to Access Files in S3

Via AWS IAM (Console)

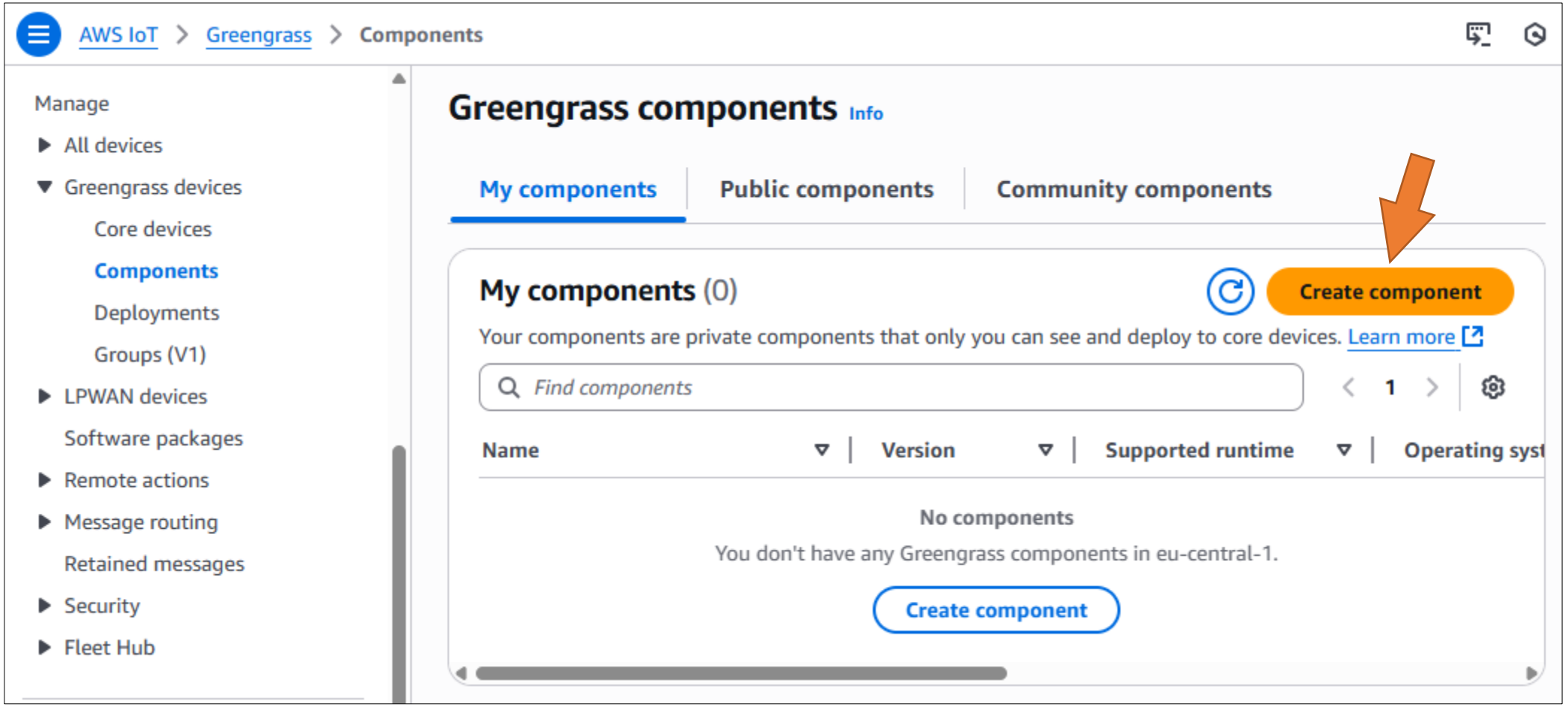
- You can create a new policy or simply attach an inline policy to related role in order to allow core device to Access objects in the S3 bucket.





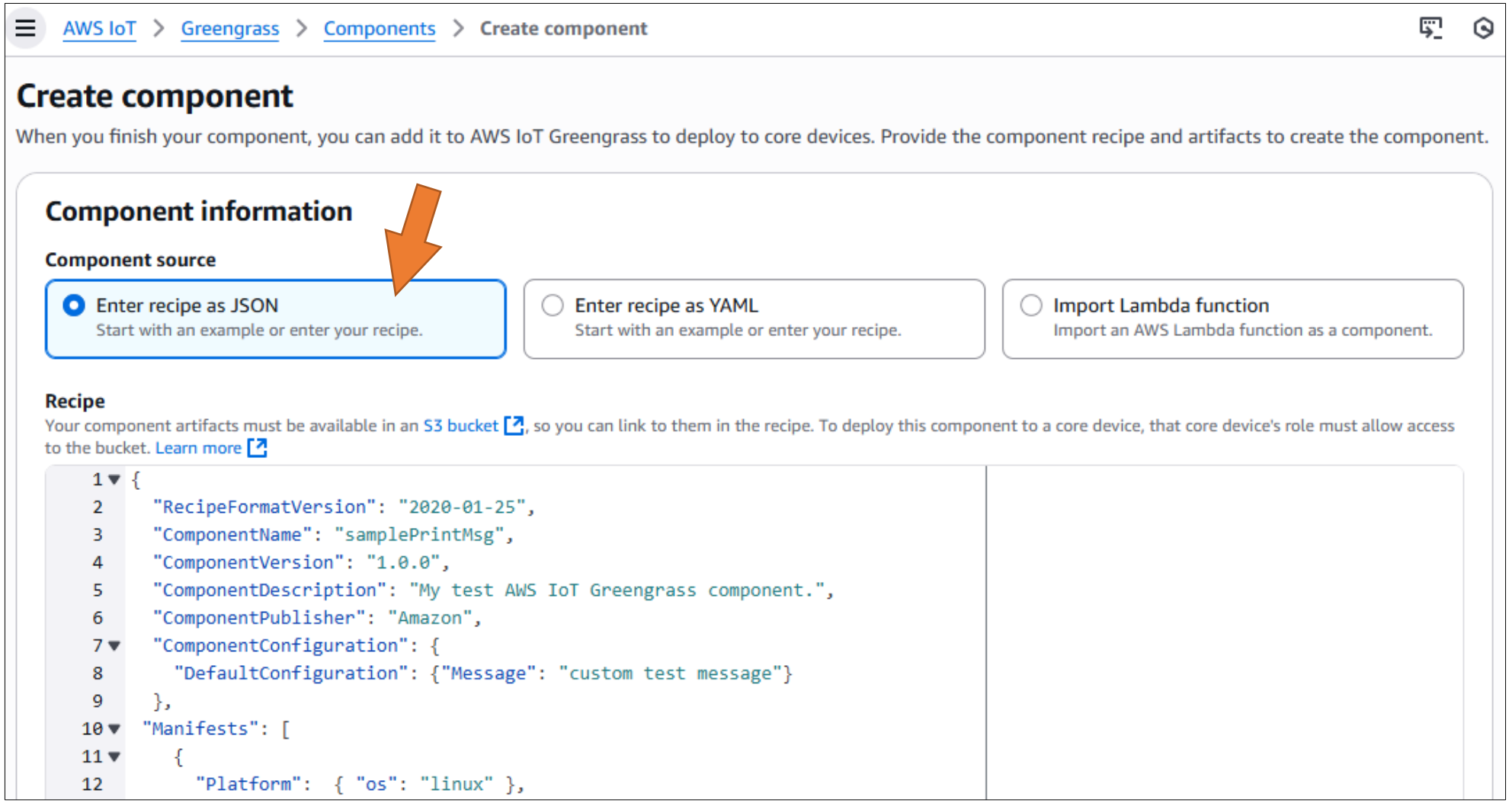
Step 3: Create the Component

Via AWS IoT Greengrass (Console)



Step 3: Create the Component

Define the Recipe



Step 3: Create the Component

Configure the Recipe

```
{
 "RecipeFormatVersion": "2020-01-25",
 "ComponentName": "samplePrintMsg",
 "ComponentVersion": "1.0.0",
 "ComponentDescription": "My test AWS IoT Greengrass component.",
 "ComponentPublisher": "Amazon",
 "ComponentConfiguration": {
  "DefaultConfiguration": {"Message": "custom test message"}
 },
 "Manifests": [
  {
   "Platform":  { "os": "linux" },
   "Lifecycle": { "run": "python3 -u {artifacts:path}/print_msg.py \"{configuration:/Message}\"" },
   "Artifacts": [ { "URI": "s3://cmpe583.greengrass.bucket/artifacts/print_msg.py" } ]
  }
 ]
}
```

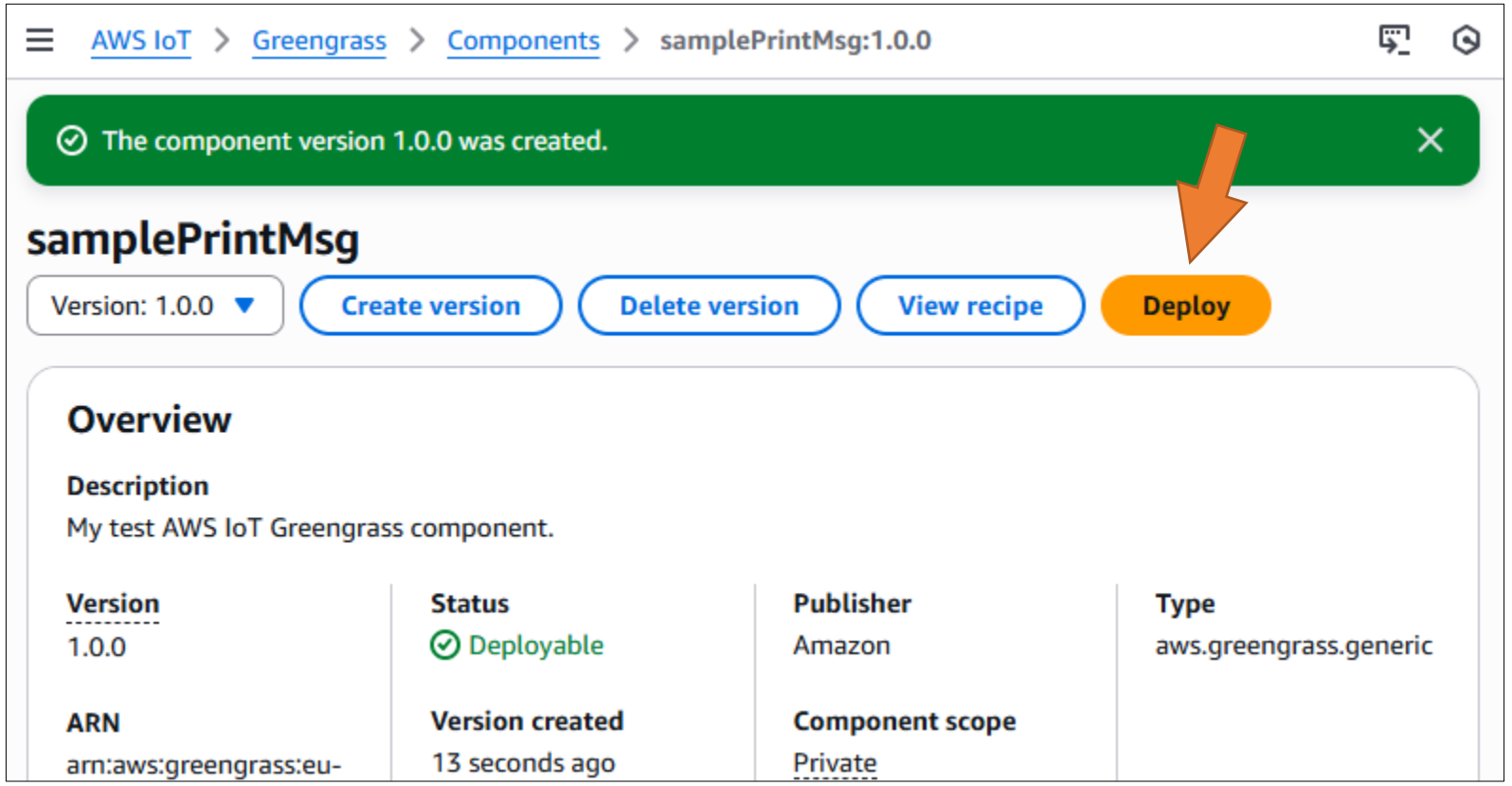
This test will be run on Linux device. So other OSs are ignored!

Define how to execute your artifact

Provide the artifact URI

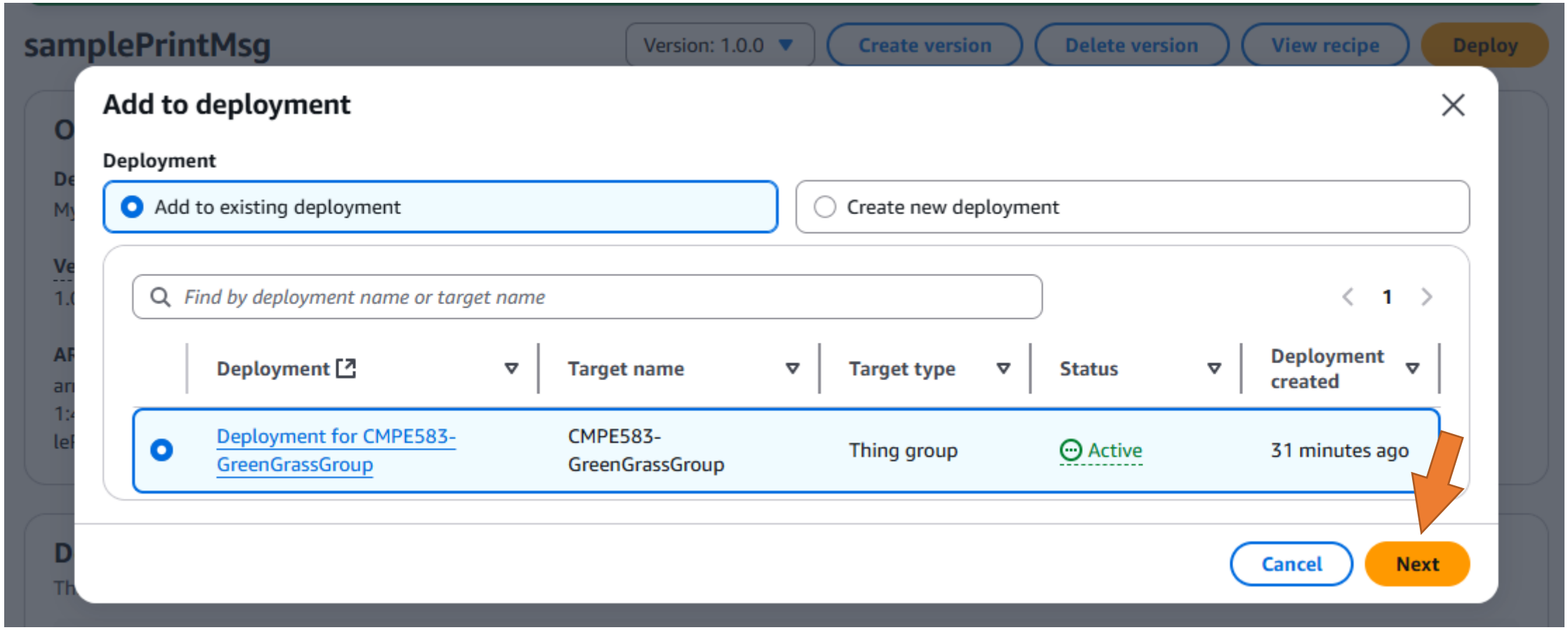
Step 4: Deploy the Component

Via AWS IoT Greengrass (console)



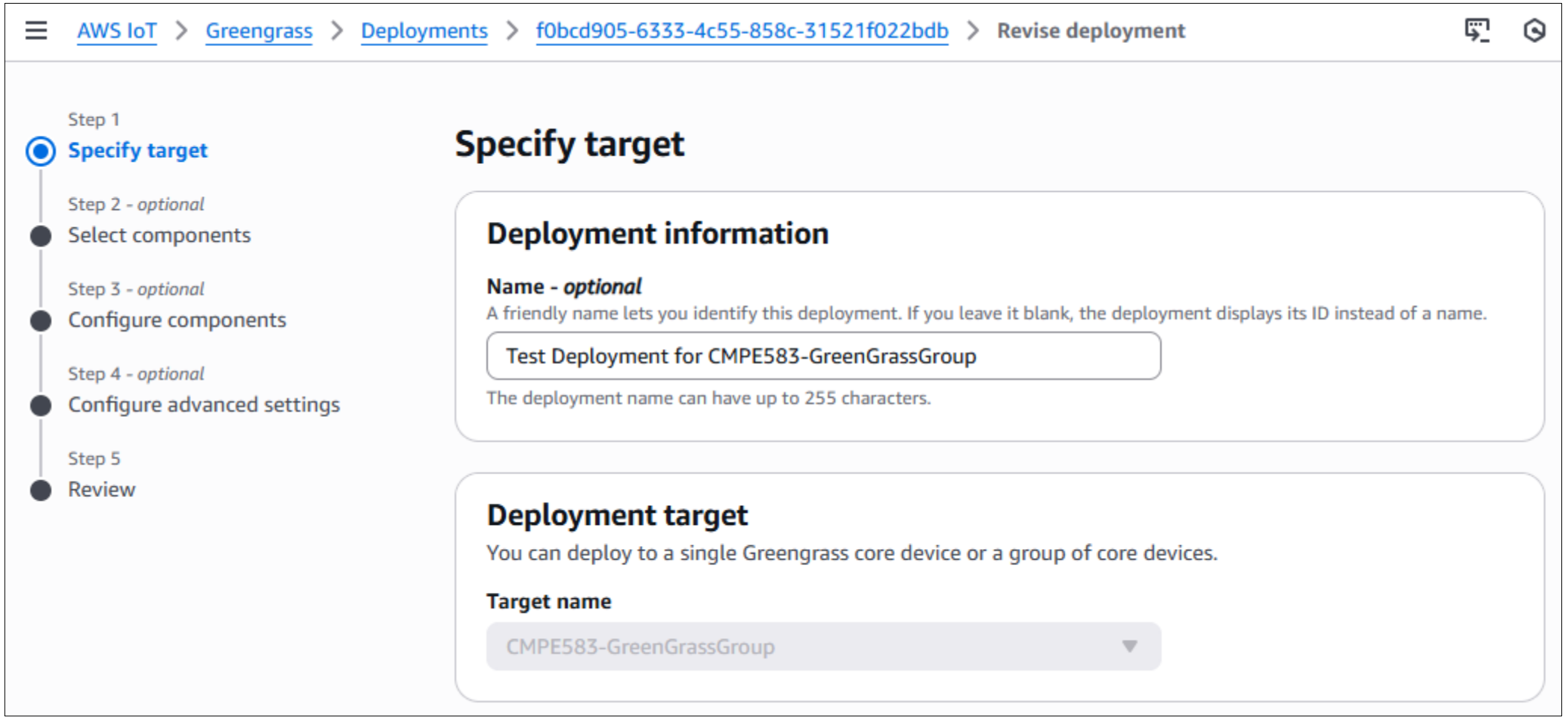
Step 4: Deploy the Component

Via AWS IoT Greengrass (console)



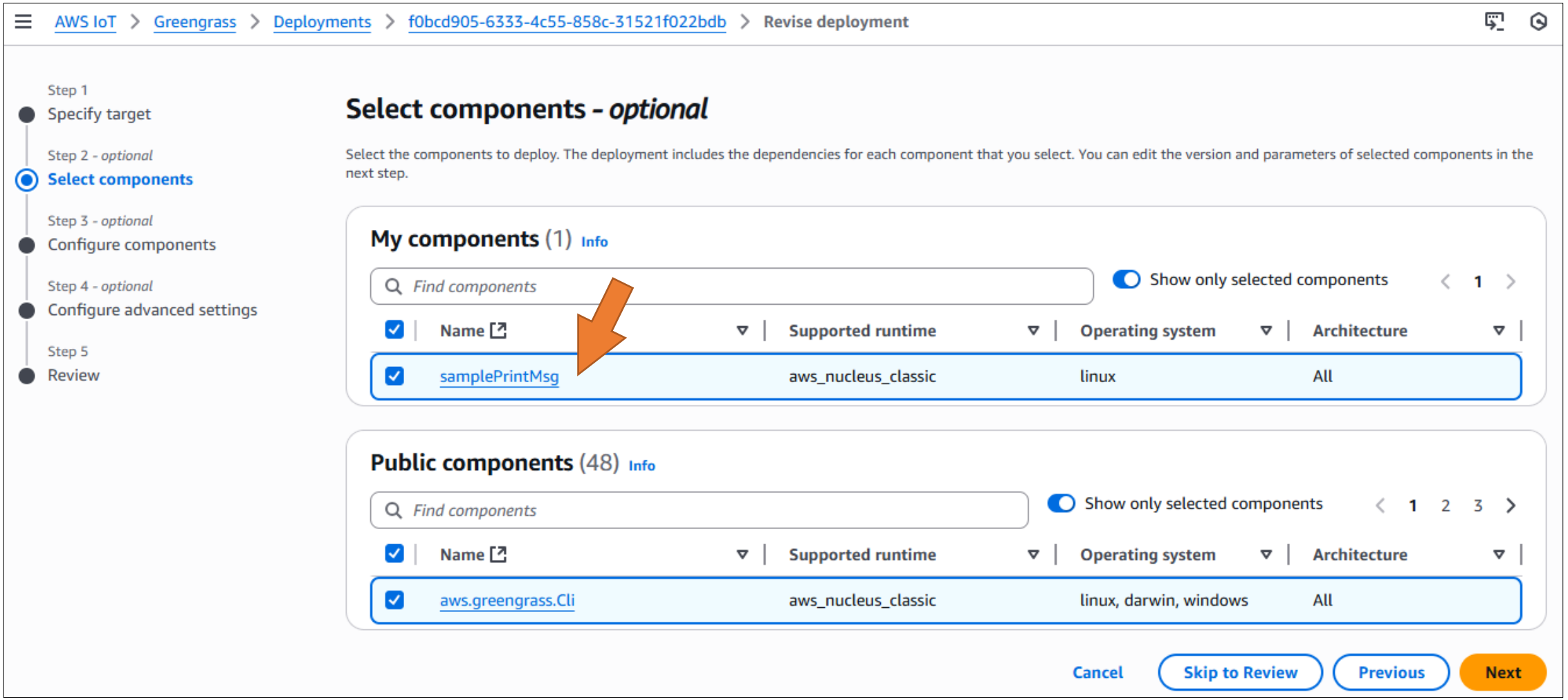
Step 4: Deploy the Component

Configure Deployment



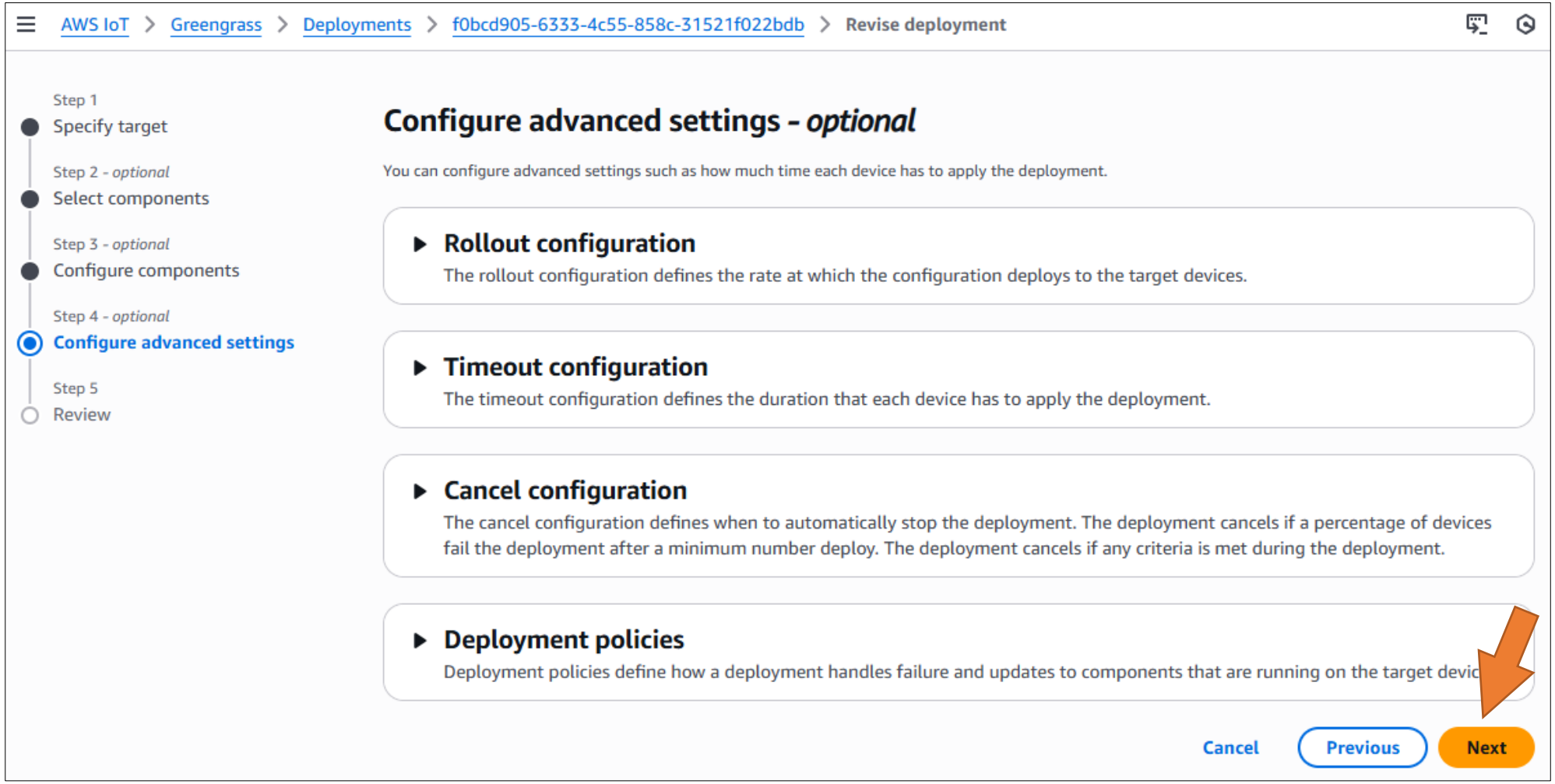
Step 4: Deploy the Component

Select the Component



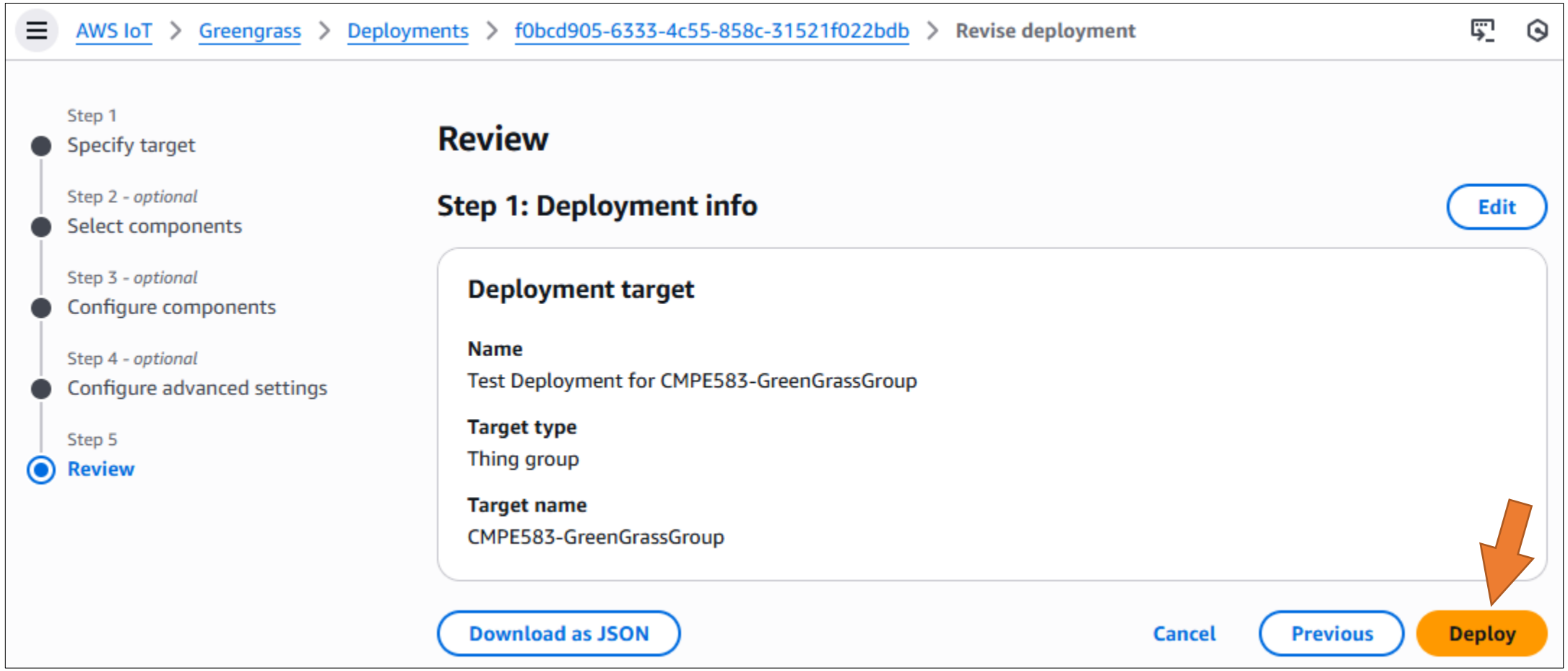
Step 4: Deploy the Component

Configure Advanced Settings



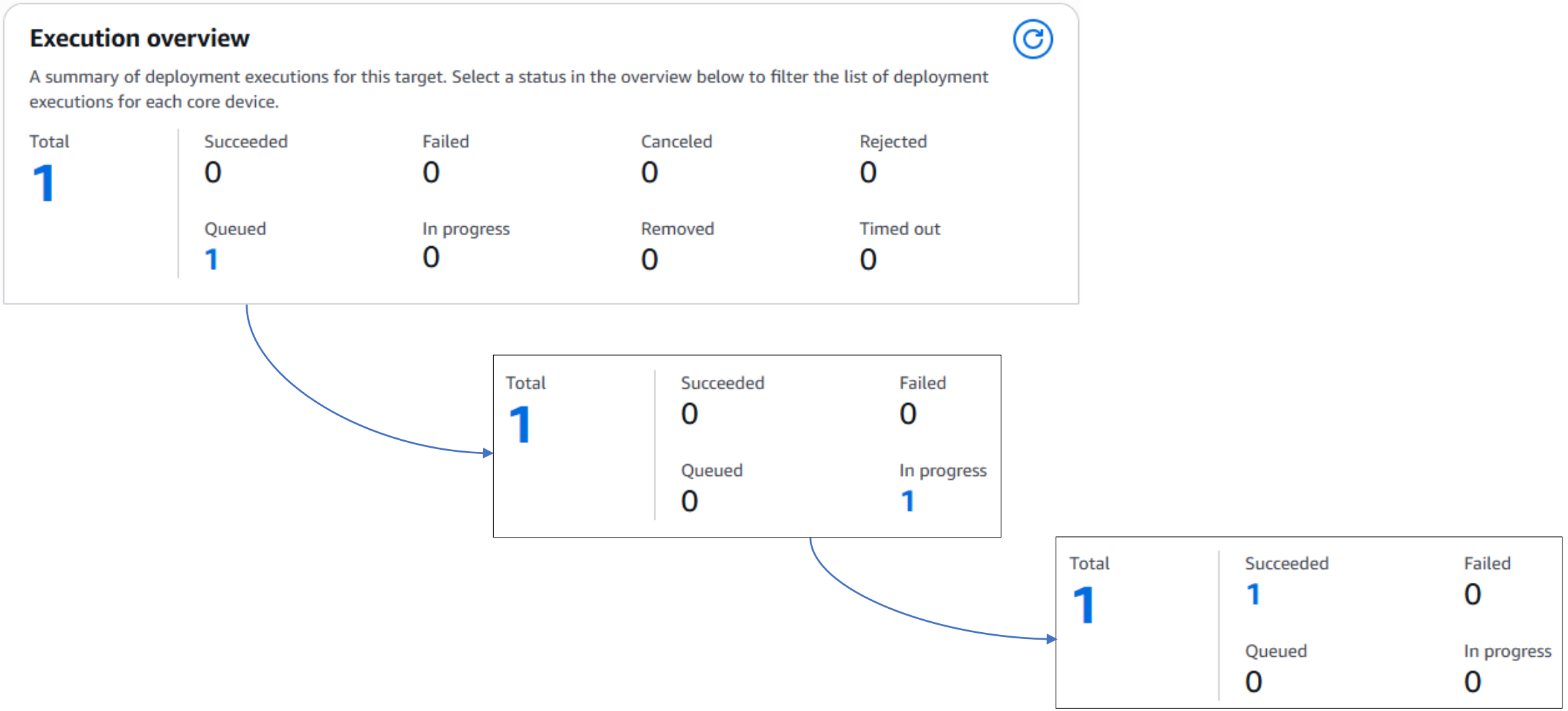
Step 4: Deploy the Component

Review & Deploy



Step 5: Verify the Component

Check AWS IoT Greengrass Console



Step 5: Verify the Component

Check Log Files

- Check the output log file created by your component to verify everything goes well.

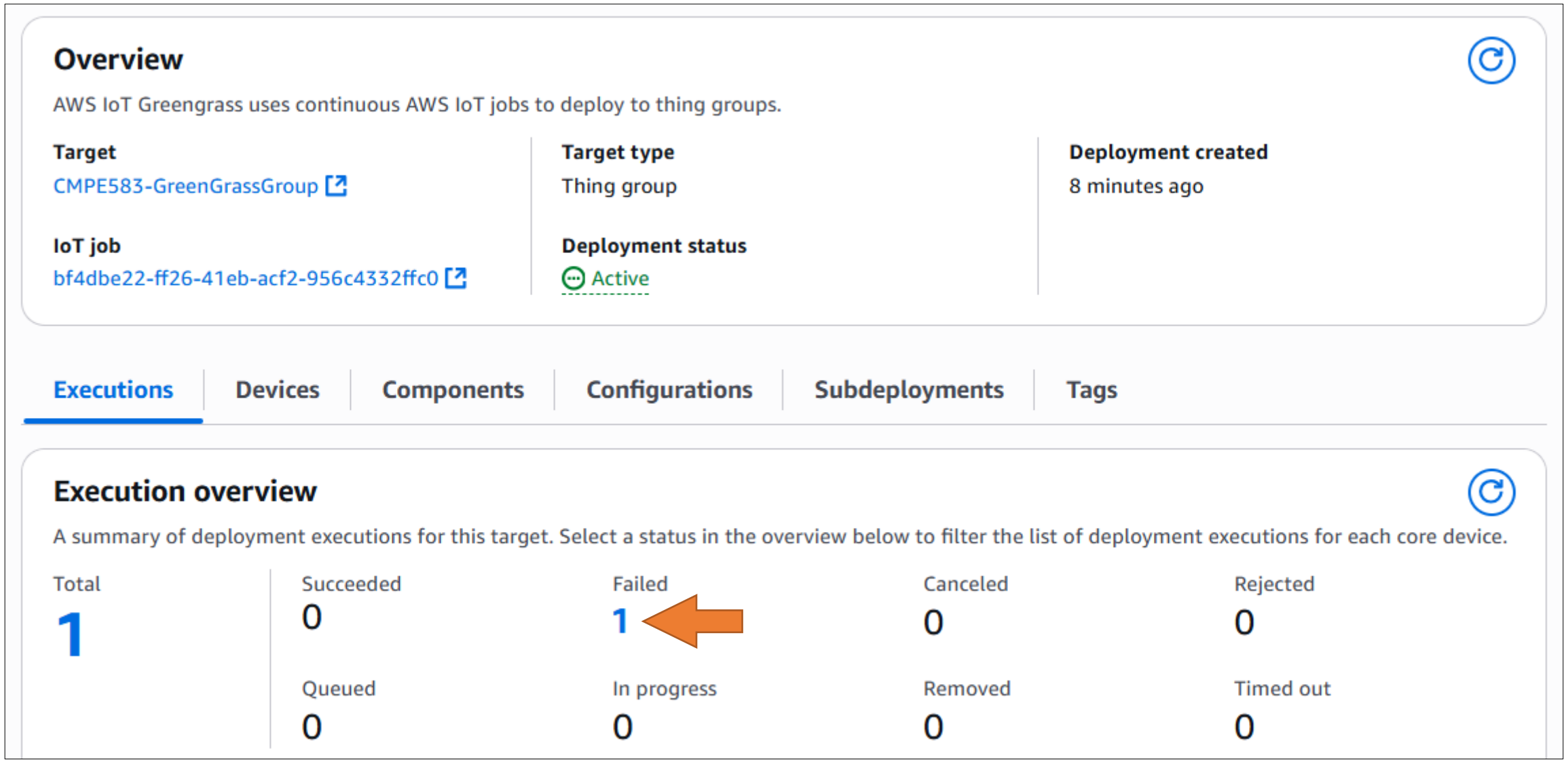
```
root@kubuntu:/home/cagatay/Projects/greengrass-demo#
root@kubuntu:/home/cagatay/Projects/greengrass-demo#
root@kubuntu:/home/cagatay/Projects/greengrass-demo# ls /greengrass/v2/logs/
aws.greengrass.Nucleus.log  greengrass_2023_10_29_15_0.log  greengrass.log  main.log  samplePrintMsg.log
root@kubuntu:/home/cagatay/Projects/greengrass-demo#
root@kubuntu:/home/cagatay/Projects/greengrass-demo#
root@kubuntu:/home/cagatay/Projects/greengrass-demo#
root@kubuntu:/home/cagatay/Projects/greengrass-demo# tail -n 100 -f /greengrass/v2/logs/samplePrintMsg.log
2023-10-29T13:04:00.263Z [INFO] (pool-2-thread-18) samplePrintMsg: shell-runner-start. {scriptName=services.samplePrintMsg.lif
ecycle.run, serviceName=samplePrintMsg, currentState=STARTING, command=["python3 -u /greengrass/v2/packages/artifacts/samplePr
intMsg/1.0.0/cmpe583-Prin..."]}
2023-10-29T13:04:00.305Z [INFO] (Copier) samplePrintMsg: stdout. Hi There, your message is: custom test message!. {scriptName=
services.samplePrintMsg.lifecycle.run, serviceName=samplePrintMsg, currentState=RUNNING}
2023-10-29T13:04:00.310Z [INFO] (Copier) samplePrintMsg: Run script exited. {exitCode=0, serviceName=samplePrintMsg, currentSt
ate=RUNNING}
```



All Good!

★ A successful deployment on AWS Greengrass console does not guarantee that the application is running or healthy; always verify component status and logs.

Troubleshooting



Troubleshooting

Check greengrass.log File

- Be sure that you IAM role is sufficient to access any resources defined in the deployment!

```
2023-09-23T10:40:57.822Z [INFO] (pool-2-thread-27) com.aws.greengrass.tes.CredentialRequestHandler: Received IAM credentials that will be cached until 2023-09-23
T11:35:57Z. {iotCredentialsPath=/role-aliases/GreengrassV2TokenExchangeRoleAlias/credentials}
2023-09-23T10:40:58.441Z [ERROR] (pool-2-thread-27) com.aws.greengrass.componentmanager.ComponentManager: Failed to prepare package com.boun.cmpe583-v1.0.0. {}
com.aws.greengrass.componentmanager.exceptions.PackageDownloadException: Failed to download artifact name: 's3://greengrass.test.buckett/artifacts/print_msg.py'
for component com.boun.cmpe583-1.0.0, reason: S3 HeadObject returns 403 Access Denied. Ensure the IAM role associated with the core device has a policy granting
s3:GetObject
        at com.aws.greengrass.componentmanager.builtins.S3Downloader.getDownloadSize(S3Downloader.java:171)
        at com.aws.greengrass.componentmanager.ComponentManager.prepareArtifacts(ComponentManager.java:441)
        at com.aws.greengrass.componentmanager.ComponentManager.preparePackage(ComponentManager.java:397)
        at com.aws.greengrass.componentmanager.ComponentManager.lambda$preparePackages$1(ComponentManager.java:358)
        at java.base/java.util.concurrent.FutureTask.run(FutureTask.java:264)
        at java.base/java.util.concurrent.ThreadPoolExecutor.runWorker(ThreadPoolExecutor.java:1128)
        at java.base/java.util.concurrent.ThreadPoolExecutor$Worker.run(ThreadPoolExecutor.java:628)
        at java.base/java.lang.Thread.run(Thread.java:829)
Caused by: software.amazon.awssdk.services.s3.model.S3Exception: null (Service: S3, Status Code: 403, Request ID: BVCVFQ32XV9H149X, Extended Request ID: jlg8Vcxx
9IKjUwa/zzbgfRR/ujoTbrA9ZUoYZvrwqSQY7z8LN4vjtzdD+yD+ZqTXs0dUqHV65uA=)
        at software.amazon.awssdk.protocols.xml.internal.unmarshall.AwsXmlPredicatedResponseHandler.handleErrorResponse(AwsXmlPredicatedResponseHandler.java:156)
        at software.amazon.awssdk.protocols.xml.internal.unmarshall.AwsXmlPredicatedResponseHandler.handleResponse(AwsXmlPredicatedResponseHandler.java:108)
        at software.amazon.awssdk.protocols.xml.internal.unmarshall.AwsXmlPredicatedResponseHandler.handle(AwsXmlPredicatedResponseHandler.java:85)
        at software.amazon.awssdk.protocols.xml.internal.unmarshall.AwsXmlPredicatedResponseHandler.handle(AwsXmlPredicatedResponseHandler.java:43)
        at software.amazon.awssdk.awscore.client.handler.AwsSyncClientHandler$Crc32ValidationResponseHandler.handle(AwsSyncClientHandler.java:95)
        at software.amazon.awssdk.core.internal.handler.BaseClientHandler.lambda$successTransformationResponseHandler$7(BaseClientHandler.java:264)
        at software.amazon.awssdk.core.internal.http.pipeline.stages.HandleResponseStage.execute(HandleResponseStage.java:40)
        at software.amazon.awssdk.core.internal.http.pipeline.stages.HandleResponseStage.execute(HandleResponseStage.java:30)
        at software.amazon.awssdk.core.internal.http.pipeline.RequestPipelineBuilder$ComposingRequestPipelineStage.execute(RequestPipelineBuilder.java:206)
        at software.amazon.awssdk.core.internal.http.pipeline.stages.ApiCallAttemptTimeoutTrackingStage.execute(ApiCallAttemptTimeoutTrackingStage.java:73)
        at software.amazon.awssdk.core.internal.http.pipeline.stages.ApiCallAttemptTimeoutTrackingStage.execute(ApiCallAttemptTimeoutTrackingStage.java:42)
```

AWS Provided Components

AWS Provided Components

Nucleus

- AWS IoT Greengrass provides and maintains prebuilt components that you can deploy to your devices.
- Several AWS-provided components depend on specific minor versions of the nucleus.
- The nucleus is a mandatory component and the minimum requirement to run the AWS IoT Greengrass Core software on a device.
- It manages deployments, orchestration, and lifecycle management of other components.
- You need to update these components when you update the Greengrass nucleus to a new minor version.

AWS Provided Components

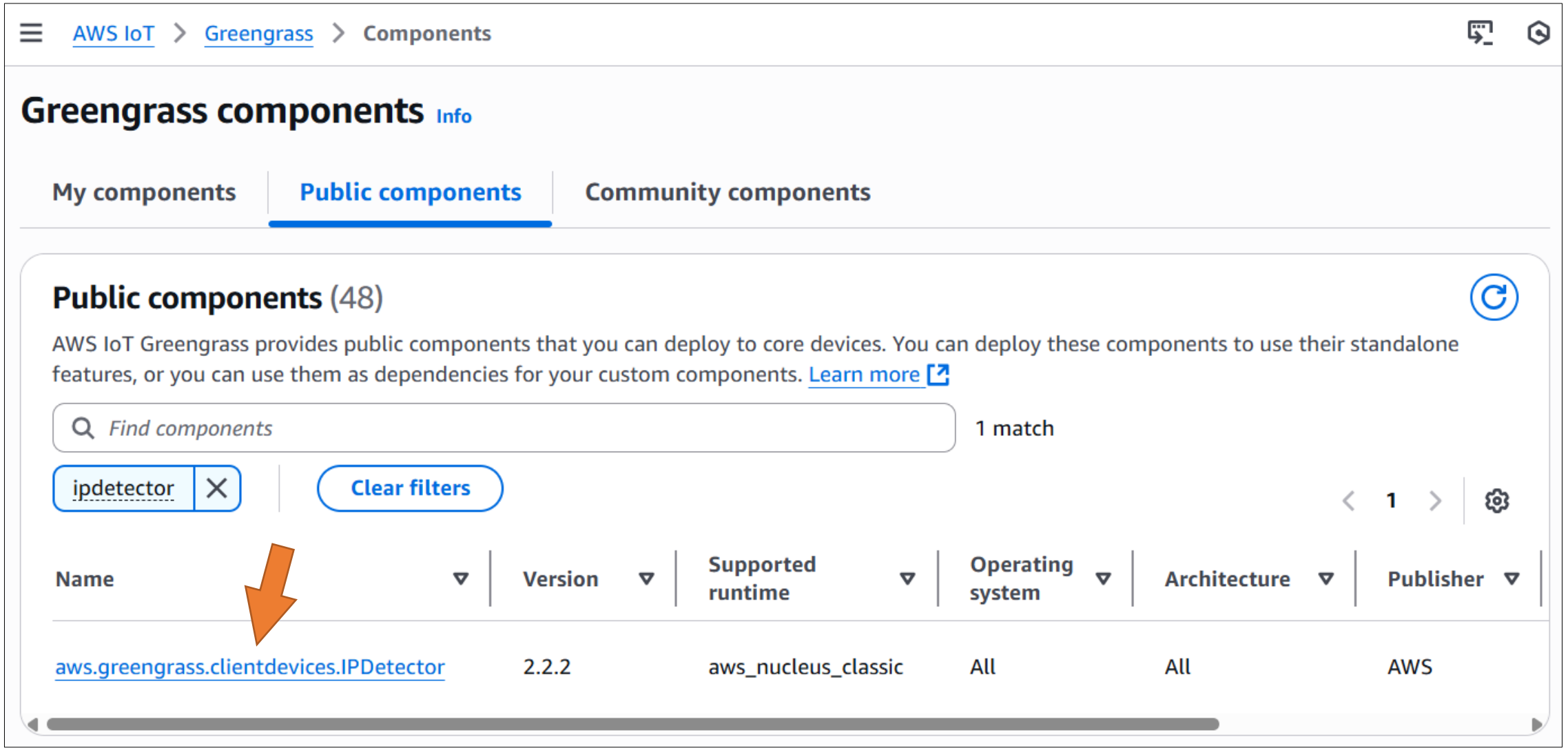
OS Support and More



Component	Description	Depends on nucleus	Component type	Supported OS	Open source
Greengrass nucleus	The nucleus of the AWS IoT Greengrass Core software. Use this component to configure and update the software on your core devices.	-	Nucleus	Linux, Windows	Yes
Client device auth	Enables local IoT devices, called client devices, to connect to the core device.	**Yes**	Plugin	Linux, Windows	Yes
CloudWatch metrics	Publishes custom metrics to Amazon CloudWatch.	**Yes**	Generic, Lambda	Linux, Windows	Yes
AWS IoT Device Defender	Notifies administrators of changes in the state of the Greengrass core device to identify unusual behavior.	**Yes**	Generic, Lambda	Linux, Windows	Yes
Disk spooler	Enables a persistent storage option for messages spooled from Greengrass core devices to AWS IoT Core. This component will store these outbound messages on disk.	**Yes**	Plugin	Linux, Windows	Yes
Docker application manager	Enables AWS IoT Greengrass to download Docker images from Docker Hub and Amazon Elastic Container Registry (Amazon ECR).	**Yes**	Generic	Linux, Windows	No
Edge connector for Kinesis Video Streams	Reads video feeds from local cameras, publishes the streams to Kinesis Video Streams, and displays the streams in Grafana dashboards with AWS IoT TwinMaker.	**Yes**	Generic	Linux	No
Greengrass CLI	Provides a command-line interface that you can use to create local deployments and interact with the Greengrass core device and its components.	**Yes**	Plugin	Linux, Windows	Yes
IP detector	Reports MQTT broker connectivity information to AWS IoT Greengrass, so client devices can discover how to connect.	**Yes**	Plugin	Linux, Windows	Yes
Kinesis Data Firehose	Publishes data through Amazon Kinesis Data Firehose delivery streams to destinations in the AWS Cloud.	**Yes**	Lambda	Linux	No
Lambda launcher	Handles processes and environment configuration for Lambda functions.	No	Generic	Linux	No
Lambda manager	Handles interprocess communication and scaling for Lambda functions.	**Yes**	Plugin	Linux	No
Lambda runtimes	Provides artifacts for each Lambda runtime.	No	Generic	Linux	No
Legacy subscription router	Manages subscriptions for Lambda functions that run on AWS IoT Greengrass V1.	**Yes**	Generic	Linux	No
Local debug console	Provides a local console that you can use to debug and manage the Greengrass core device and its components.	**Yes**	Plugin	Linux, Windows	Yes
Log manager	Collects and uploads logs on the Greengrass core device.	**Yes**	Plugin	Linux, Windows	Yes

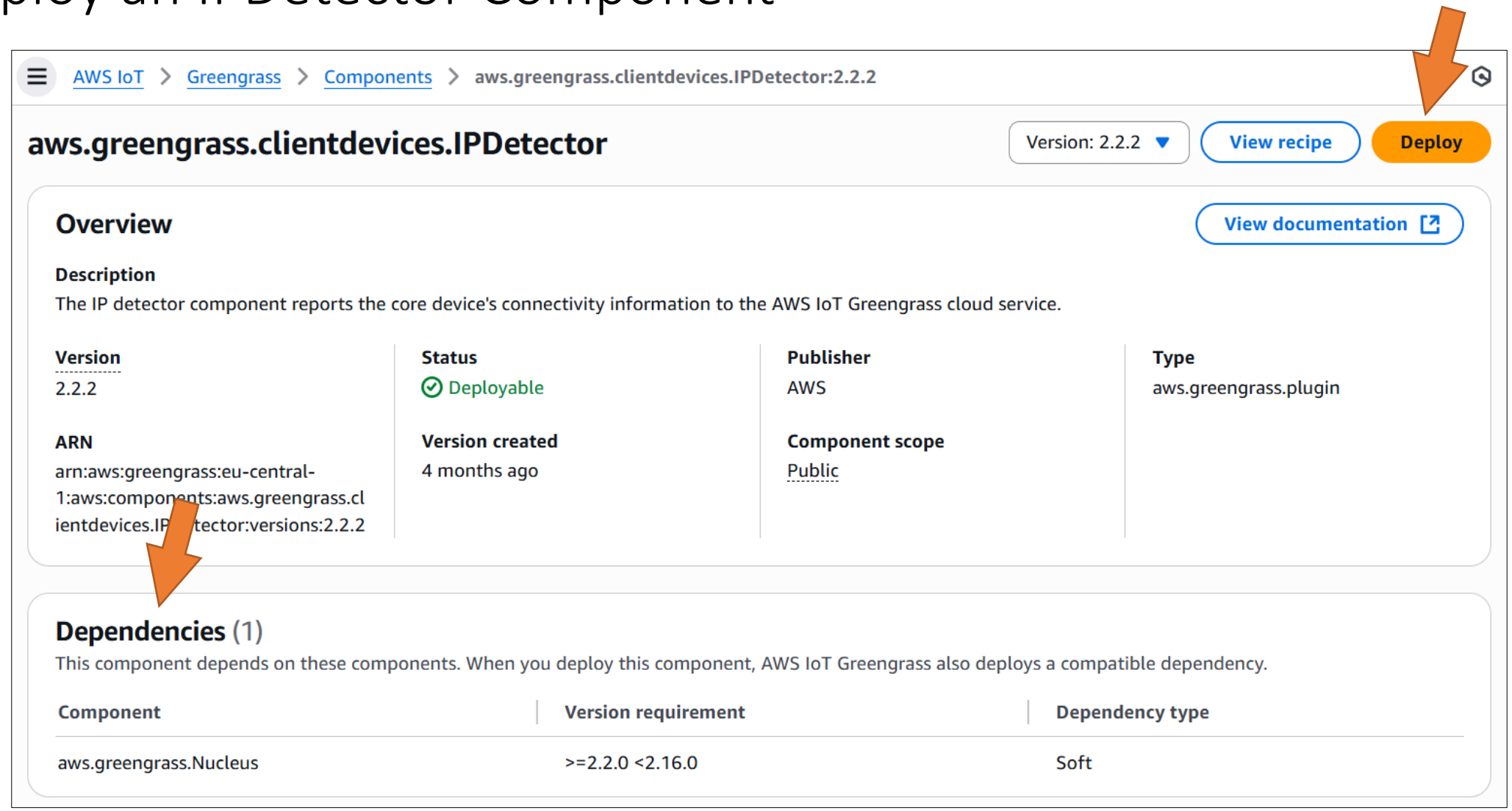
Collecting IP Addresses of Core Devices

IPDetector



Collecting IP Addresses of Core Devices

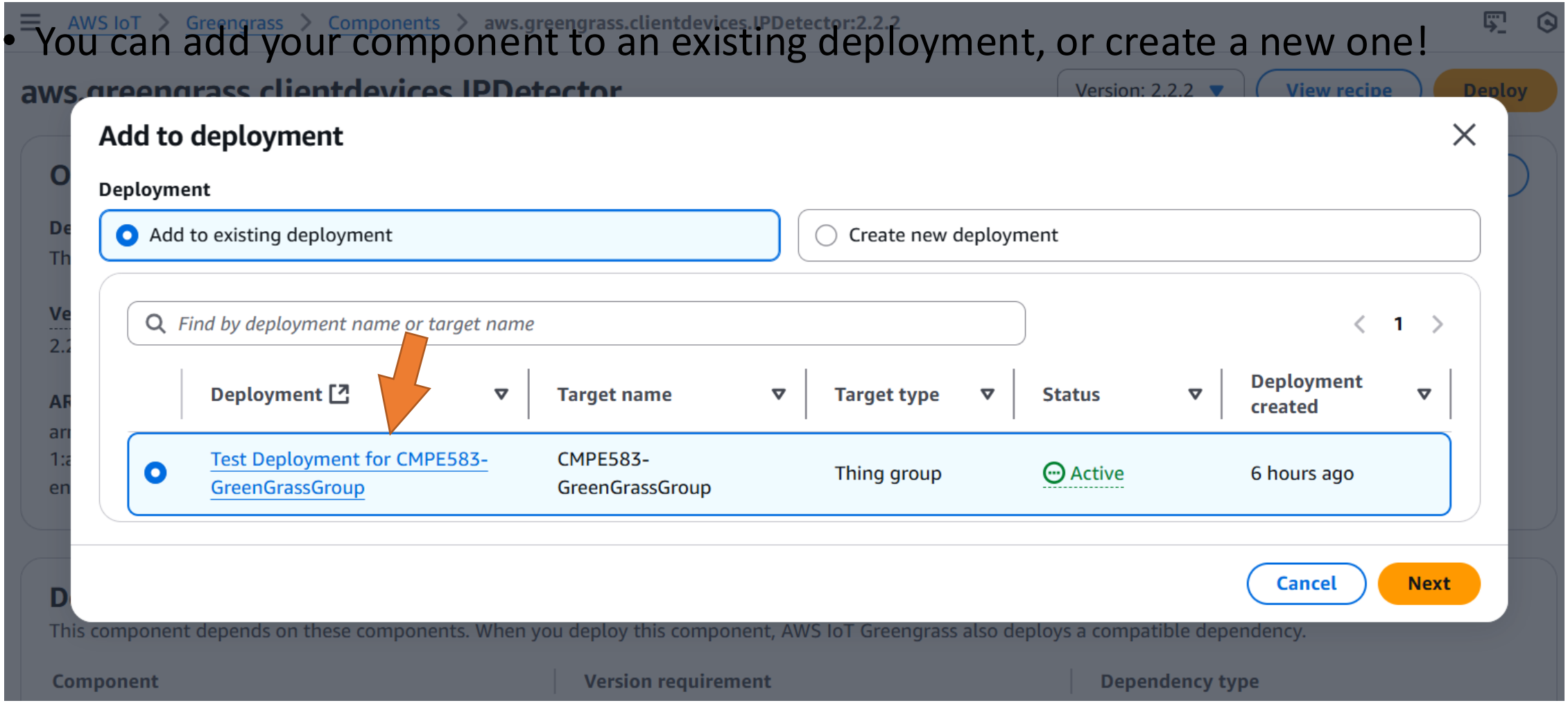
Deploy an IPDetector Component



Collecting IP Addresses of Core Devices

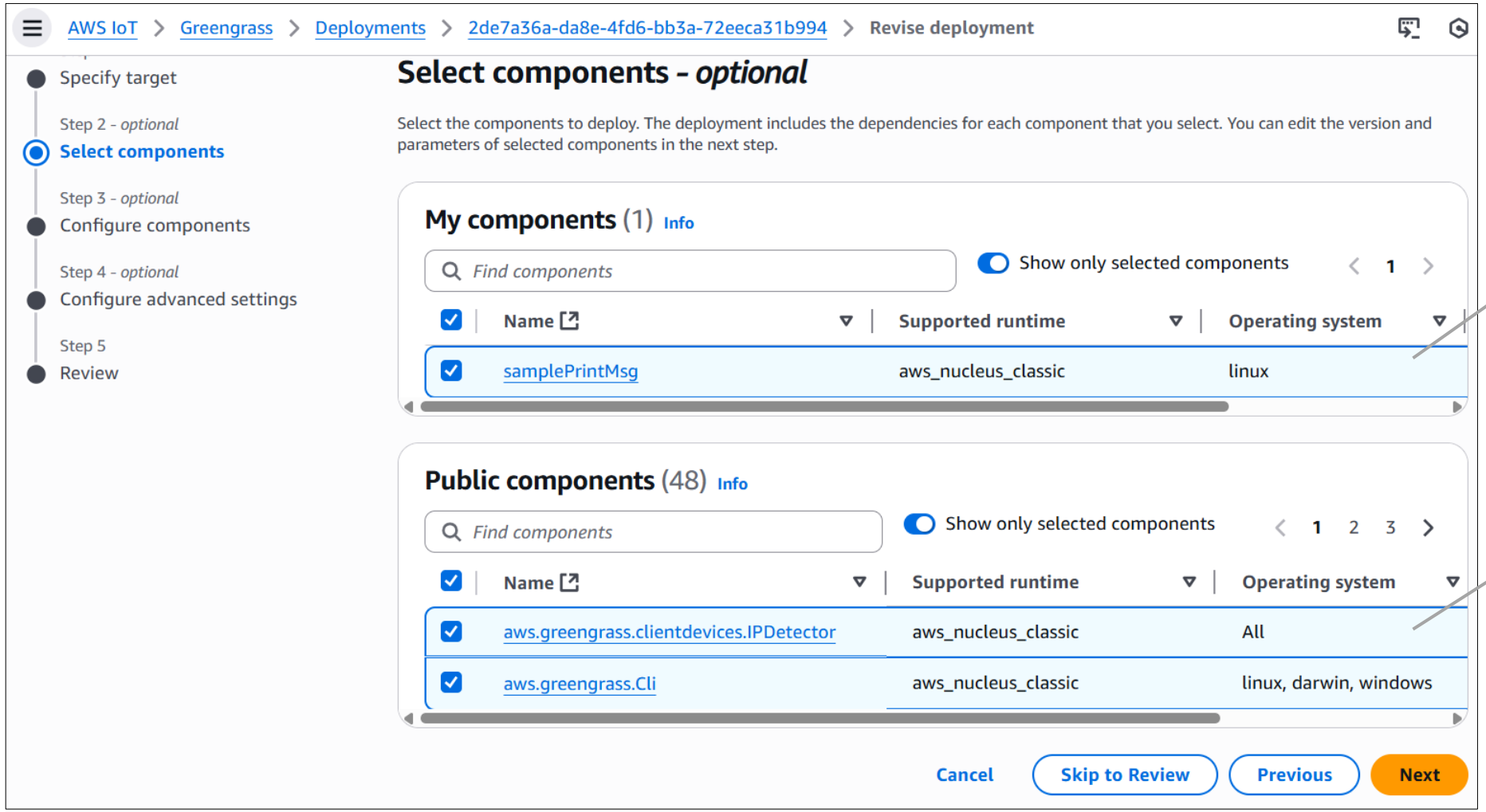
Add Public Component to Existing Deployment

- You can add your component to an existing deployment, or create a new one!



Collecting IP Addresses of Core Devices

Select All Components You Want to Deploy



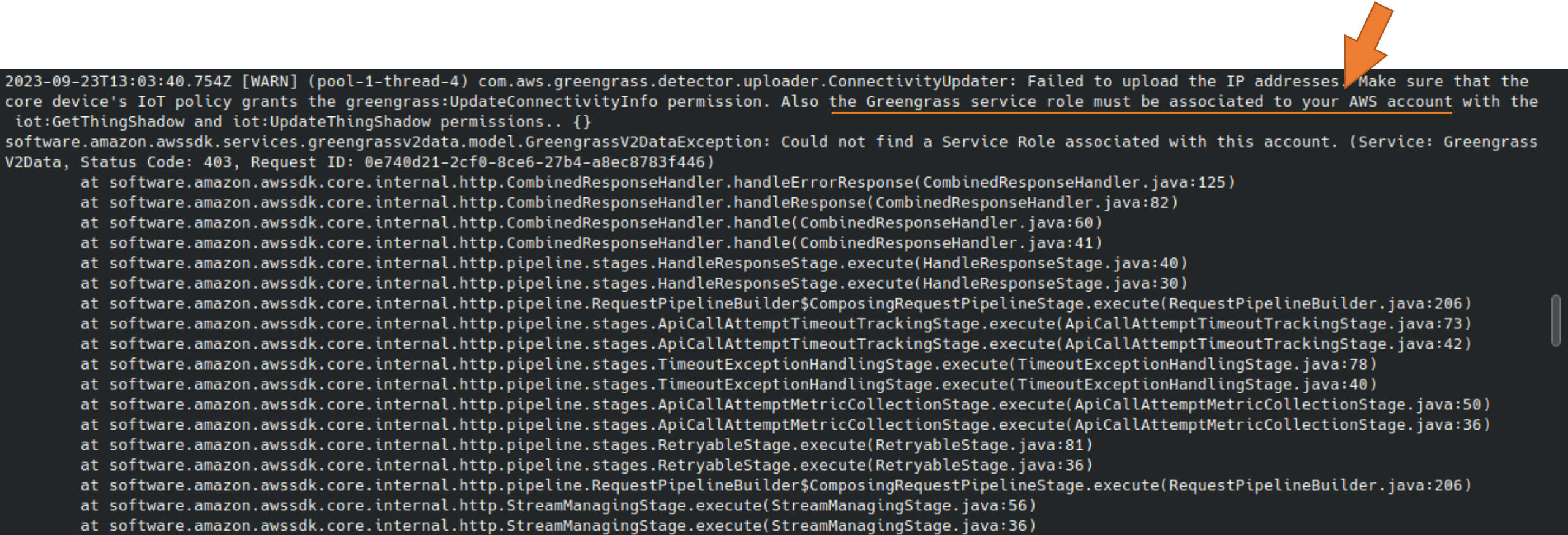
Custom component

AWS Provided component

Collecting IP Addresses of Core Devices

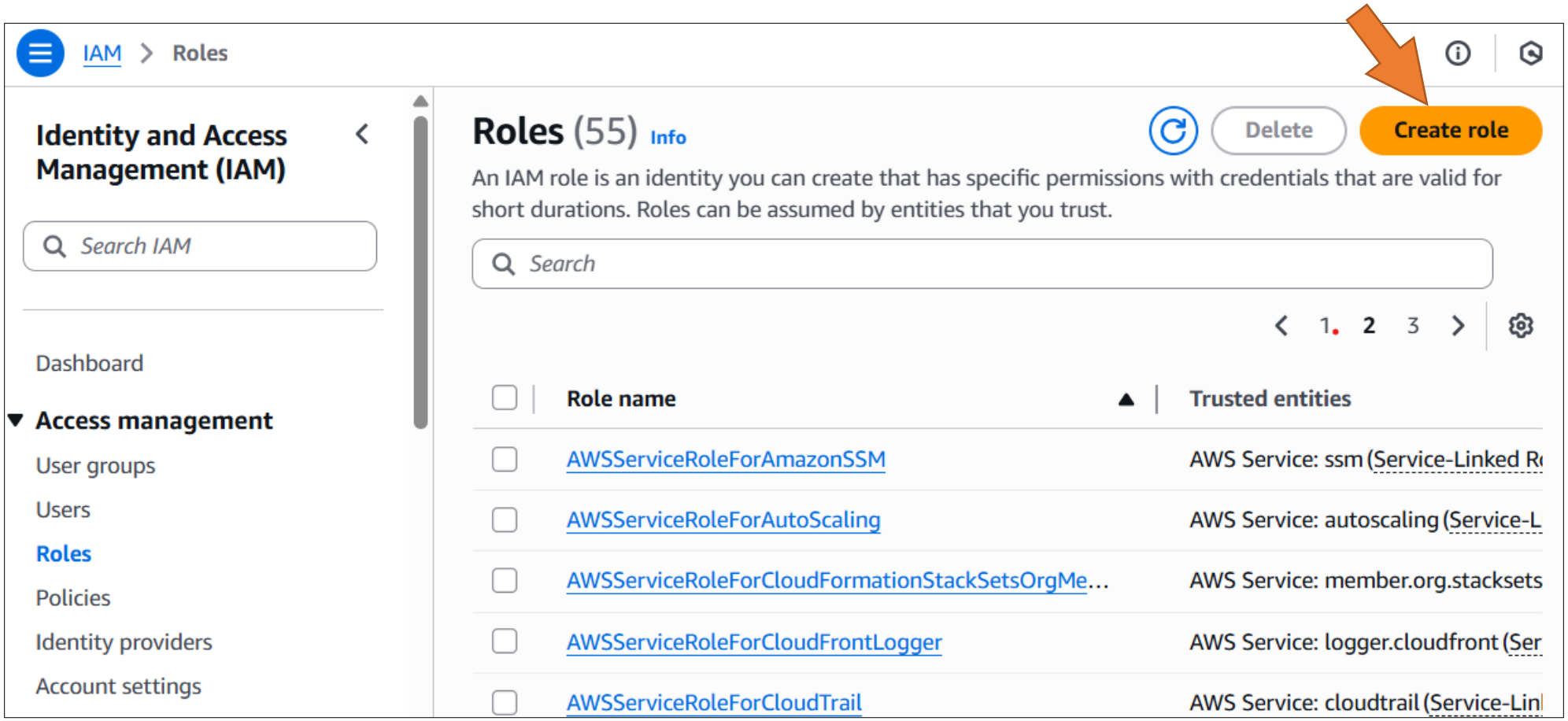
Troubleshooting

- Check greengrass.log file if the deployed job runs without error.
- You will see an error if the Greengrass service role is not associated to your AWS account.
- Greengrass needs permission to access the AWS services on your behalf.



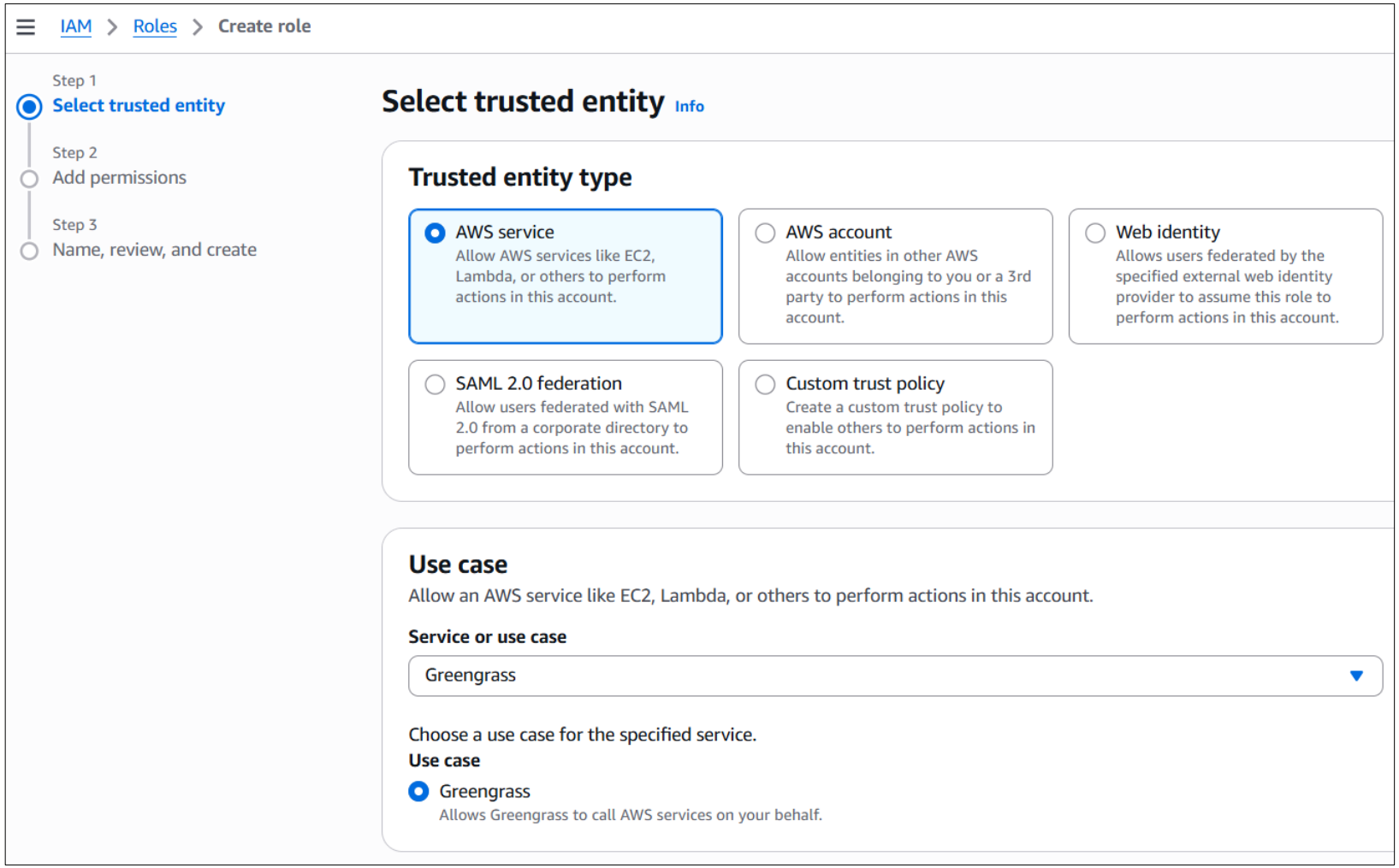
Troubleshooting

Create an IAM Role



Troubleshooting

Create an IAM Role

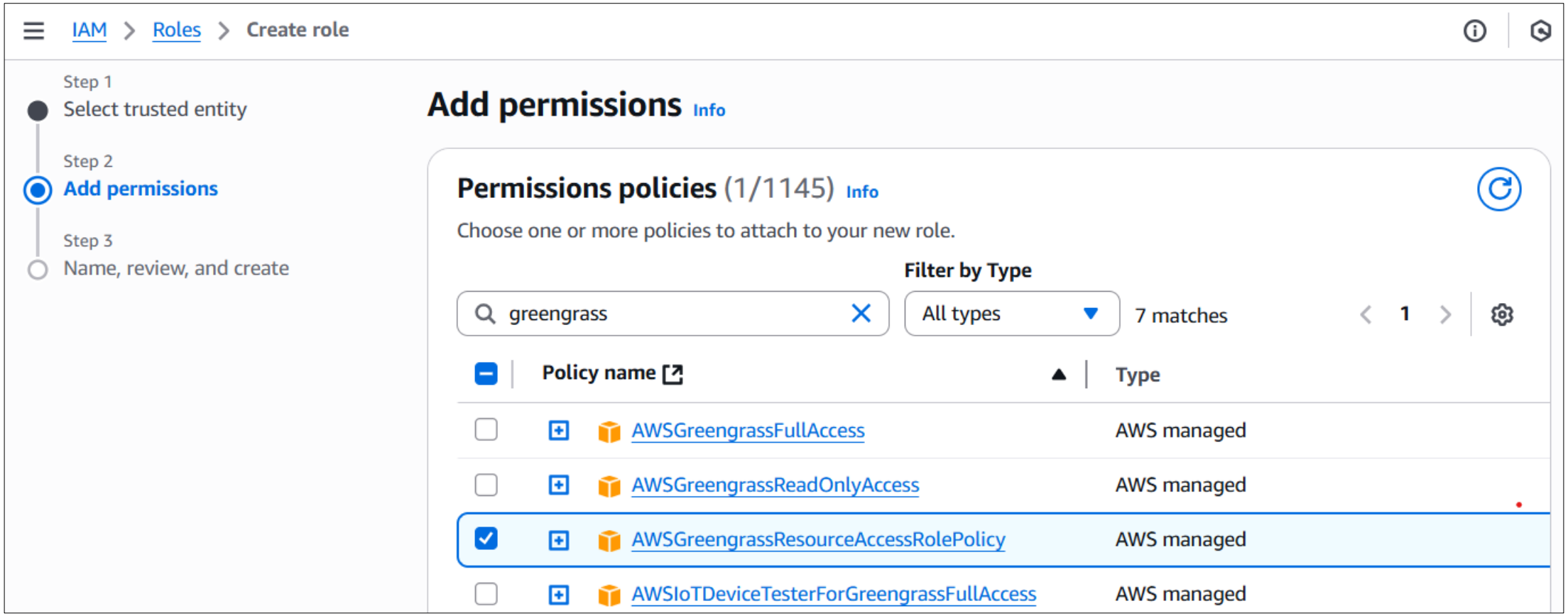


- Create a role that allows an AWS service to perform actions on your behalf.
- Allows Greengrass to call AWS services on your behalf.

Troubleshooting

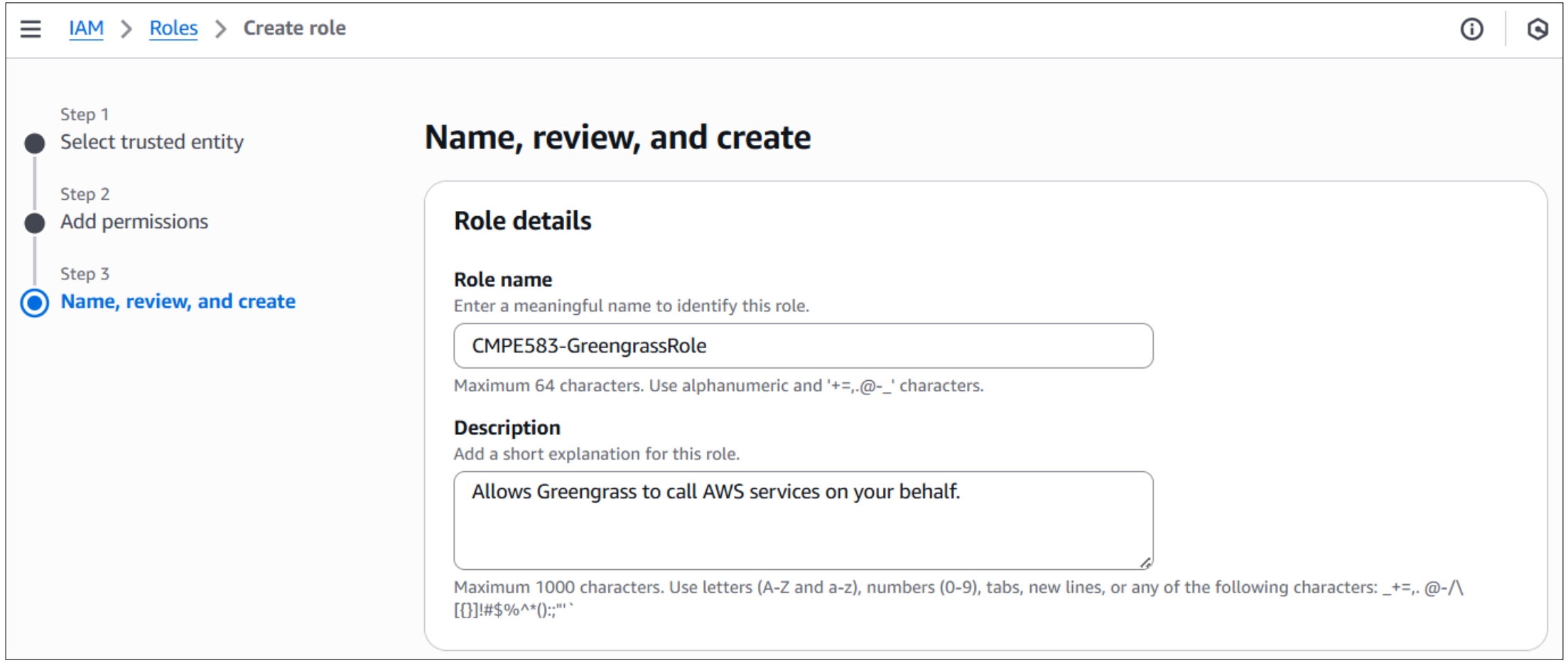
Add Permission to Created Role

- To allow AWS IoT Greengrass to access your resources, the Greengrass service role must be associated with your AWS account and specify AWS IoT Greengrass as a trusted entity.



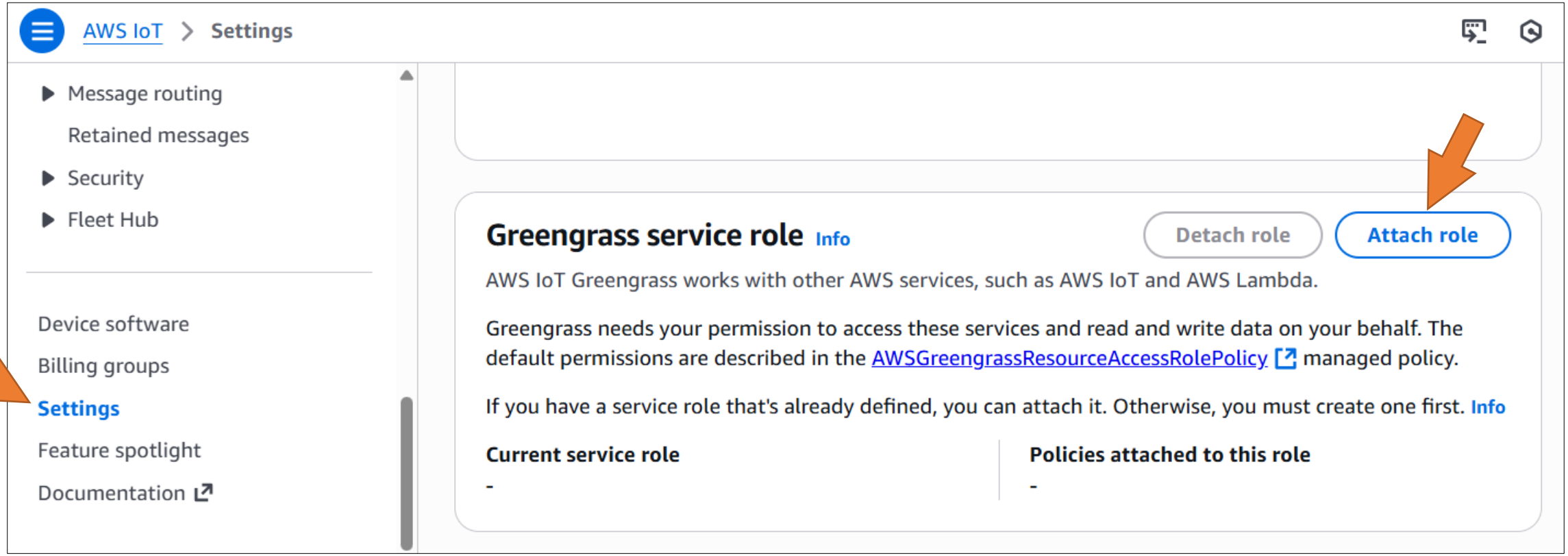
Troubleshooting

Review and Create Role



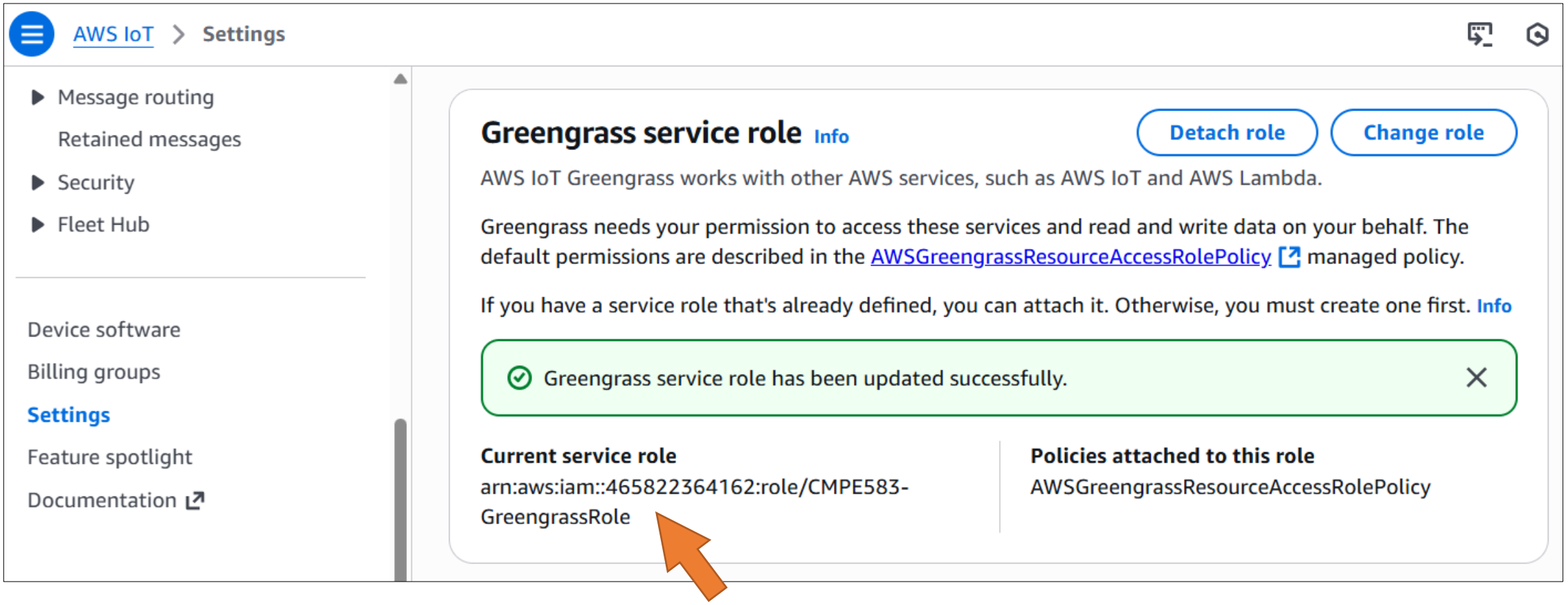
Troubleshooting

Attach Role to AWS IoT Service



Troubleshooting

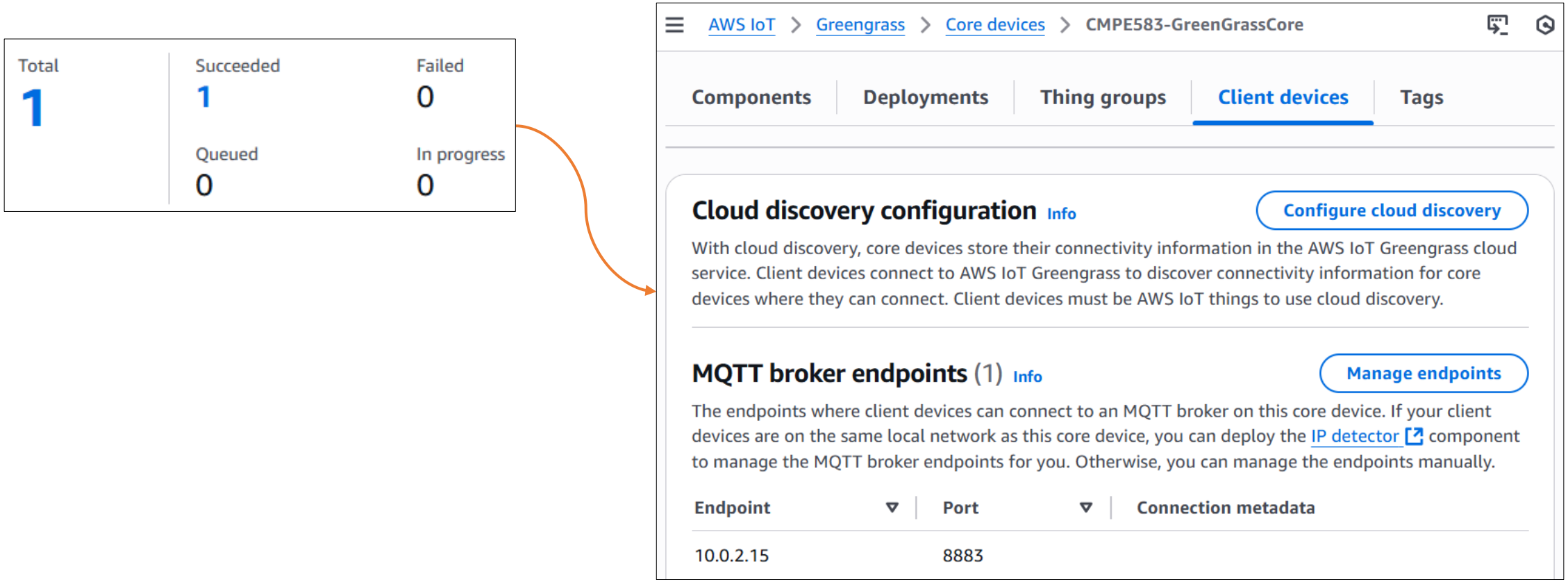
Attach Role to AWS IoT Service



Troubleshooting

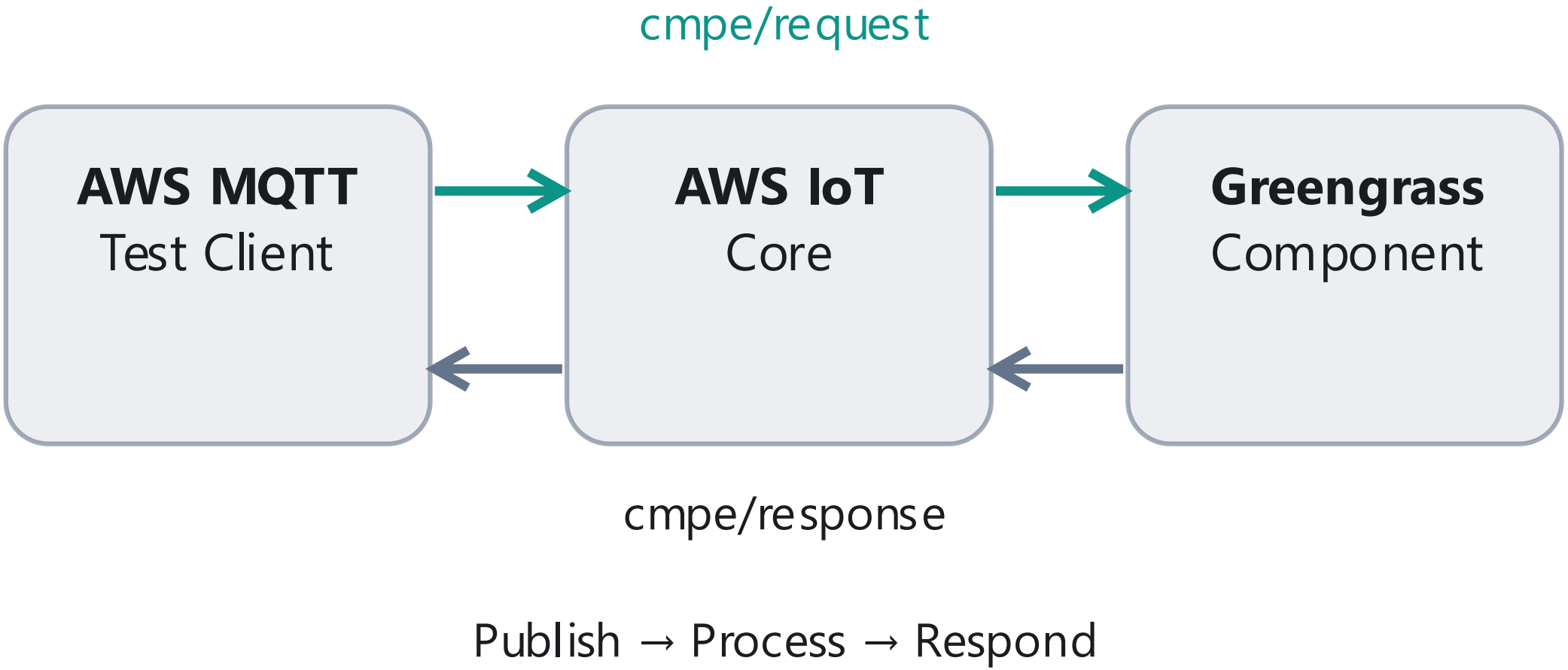
Verify the Deployment

- You can see the IP addresses of your core devices after the deployment succeeded.



Publish/Subscribe AWS IoT Core MQTT Messages

AWS IoT Core MQTT Communication Flow



Publish → Process → Respond

AWS IoT Greengrass Core IPC

- Components running on your core device can use the AWS IoT Greengrass Core interprocess communication (IPC) library in the AWS IoT Device SDK to communicate with the AWS IoT Greengrass nucleus and other Greengrass components.
- AWS IoT Greengrass provides an improved version of the IPC client: IPC client V2
- The IPC interface supports two types of operations:
  - **Request/response**
    - Components send a request to the IPC service and receive a response that contains the result of the request.
  - **Subscription**
    - Components send a subscription request to the IPC service and expect a stream of event messages in response.

AWS IoT Core MQTT Messaging IPC

- The AWS IoT Core MQTT messaging IPC service lets you send and receive MQTT messages to and from AWS IoT Core.
- Components can publish messages to AWS IoT Core and subscribe to topics to act on MQTT messages from other sources.
- To use AWS IoT Core MQTT messaging in a custom component, you must define authorization policies that allow your component to send and receive messages on topics.
  - **aws.greengrass#PublishToIoTCore**
    - Allows a component to publish messages to AWS IoT Core on the MQTT topics.
  - **aws.greengrass#SubscribeToIoTCore**
    - Allows a component to subscribe to messages from AWS IoT Core on the topics.

MQTT

- MQTT (Message Queuing Telemetry Transport) is a lightweight and widely adopted messaging protocol that is designed for constrained devices.
- AWS IoT Core support for MQTT is based on the MQTT v3.1.1 specification and the MQTT v5.0 specification.
- As the latest version of the standard, MQTT 5 introduces several key features that make an MQTT-based system more robust.
- AWS IoT Core also supports cross MQTT version (MQTT 3 and MQTT 5) communication.

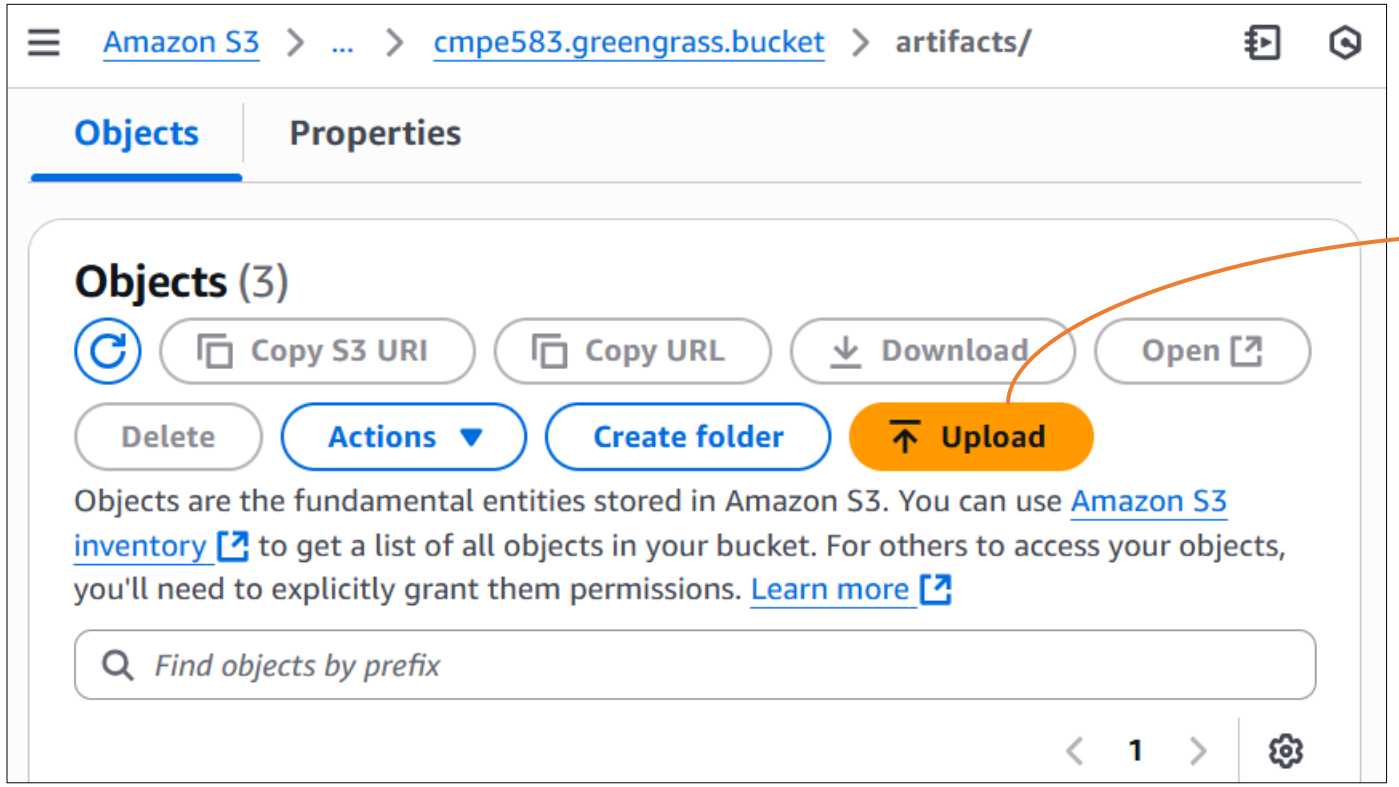
Steps for Deploying a Component Using Core IPC

1. Develop client code using Core MQTT IPC and upload artifacts to S3
2. Create the component and define its recipe
3. Set access control for MQTT topics and deploy the component
4. Verify and test the component using AWS MQTT Client

Step 1: Develop & Upload Client-Side Code

Via AWS IoT Greengrass (Console)

- Upload your artifacts to a folder in your S3 bucket



cmpe583-PubSub.py

```python
import time
import traceback
import json
import sys

from awsiot.greengrasscoreipc.clientv2 import GreengrassCoreIPCClientV2

REQUEST_TOPIC = sys.argv[1]
RESPONSE_TOPIC = sys.argv[2]

ipc_client = GreengrassCoreIPCClientV2()

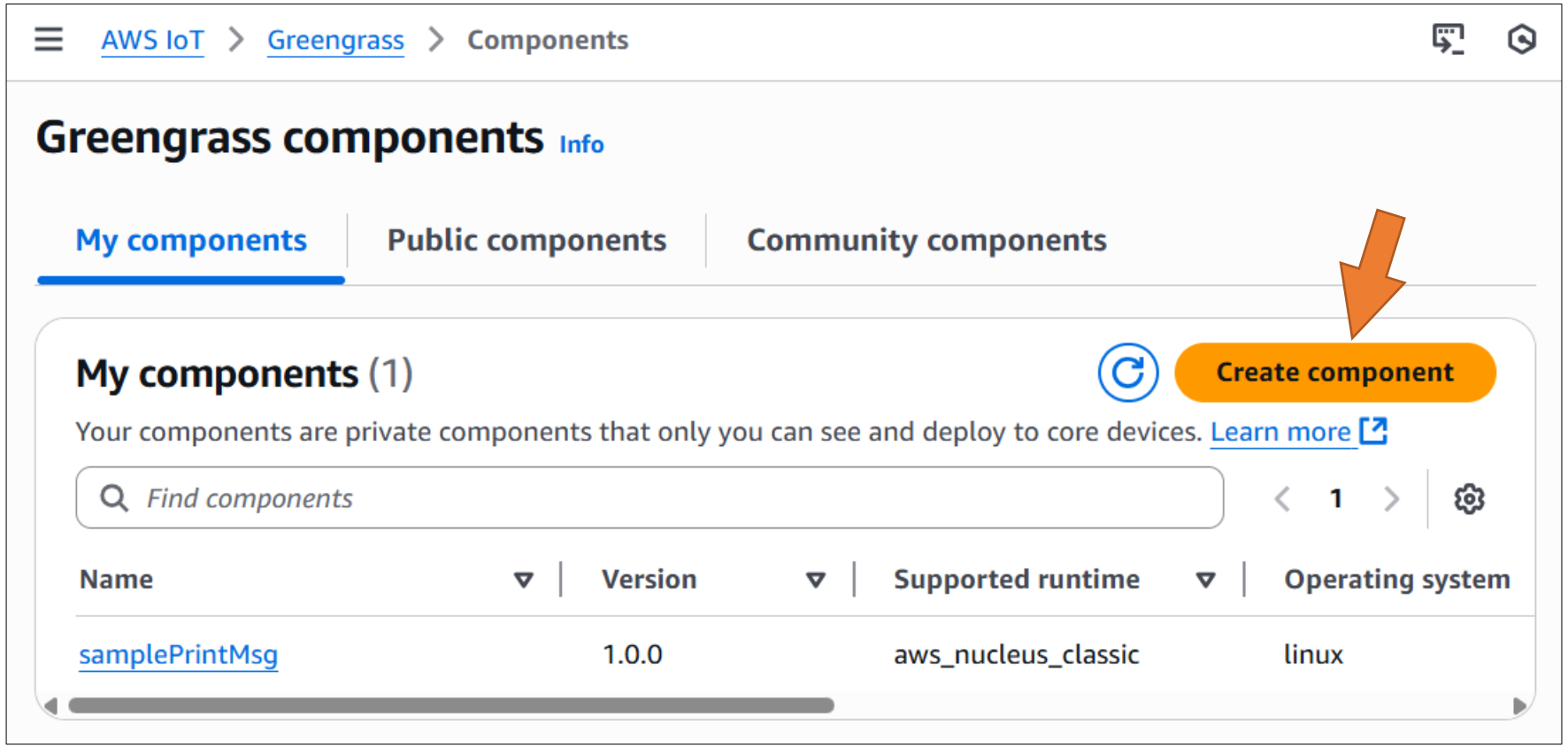

def respond(event):
    # Dummy response message
    response_message = {
        "timestamp": int(round(time.time() * 1000)),
        "response": "Hello from the Green Grass Core Device!"
    }

    # Publish to our topic
    ipc_client.publish_to_iot_core(
        topic_name=RESPONSE_TOPIC,
        payload=json.dumps(response_message).encode("utf-8"),
        qos="0"
    )


def on_stream_event(event):
    try:
        respond(event)
    except Exception:
        traceback.print_exc()
```

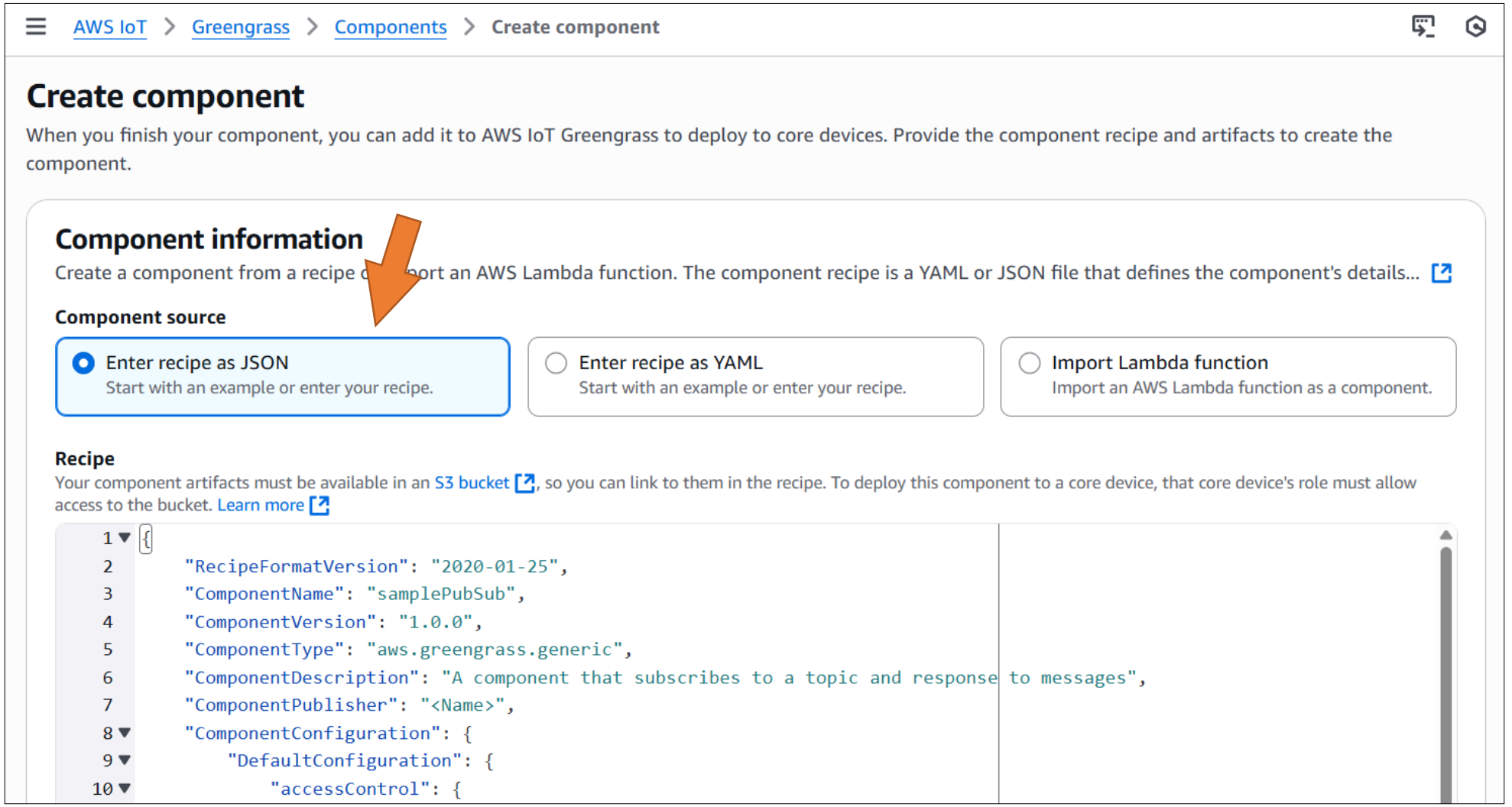
Step 2: Create a Custom Component

Via AWS IoT Greengrass (Console)



Step 2: Create a Custom Component

Enter the Recipe



Step 2: Create a Custom Component

Adjust the Access Control

```
8    "ComponentConfiguration": {
9        "DefaultConfiguration": {
10           "accessControl": {
11               "aws.greengrass.ipc.mqttproxy": {
12                   "com.example.MyIoTCorePubSubComponent:mqttproxy:1": {
13                       "policyDescription": "Allows access to publish/subscribe
14                       "operations": [
15                           "aws.greengrass#PublishToIoTCore",
16                           "aws.greengrass#SubscribeToIoTCore"
17                       ],
18                       "resources": [
19                           "*"
20                       ]
21                   }
22               }
23           }
24       }
```

Allow the component to publish messages to AWS IoT Core or subscribe to messages from AWS IoT Core.

A topic string, such as test/topic, or * to allow access to all topics. You can use MQTT topic wildcards (# and +) to match multiple resources.

Step 2: Create a Custom Component

Adjust the Access Control

```
"Manifests": [
    {
        "Lifecycle": {
            "Install": "pip3 install --user awsiotsdk botocore --break-system-packages",
            "Run": "python3 -u {artifacts:path}/cmpe583-PubSub.py cmpe/request cmpe/response\n"
        },
        "Artifacts": [
            {
                "Uri": "s3://cmpe583.greengrass.bucket/artifacts/cmpe583-PubSub.py",
                "Digest": "ZpHa6IOEaf/NST7aso8bf52AP39DNOeNtknC/5Qah/E=",
                "Algorithm": "SHA-256",
                "Unarchive": "NONE",
                "Permission": {
                    "Read": "OWNER",
                    "Execute": "NONE"
                }
            }
        ]
    }
```

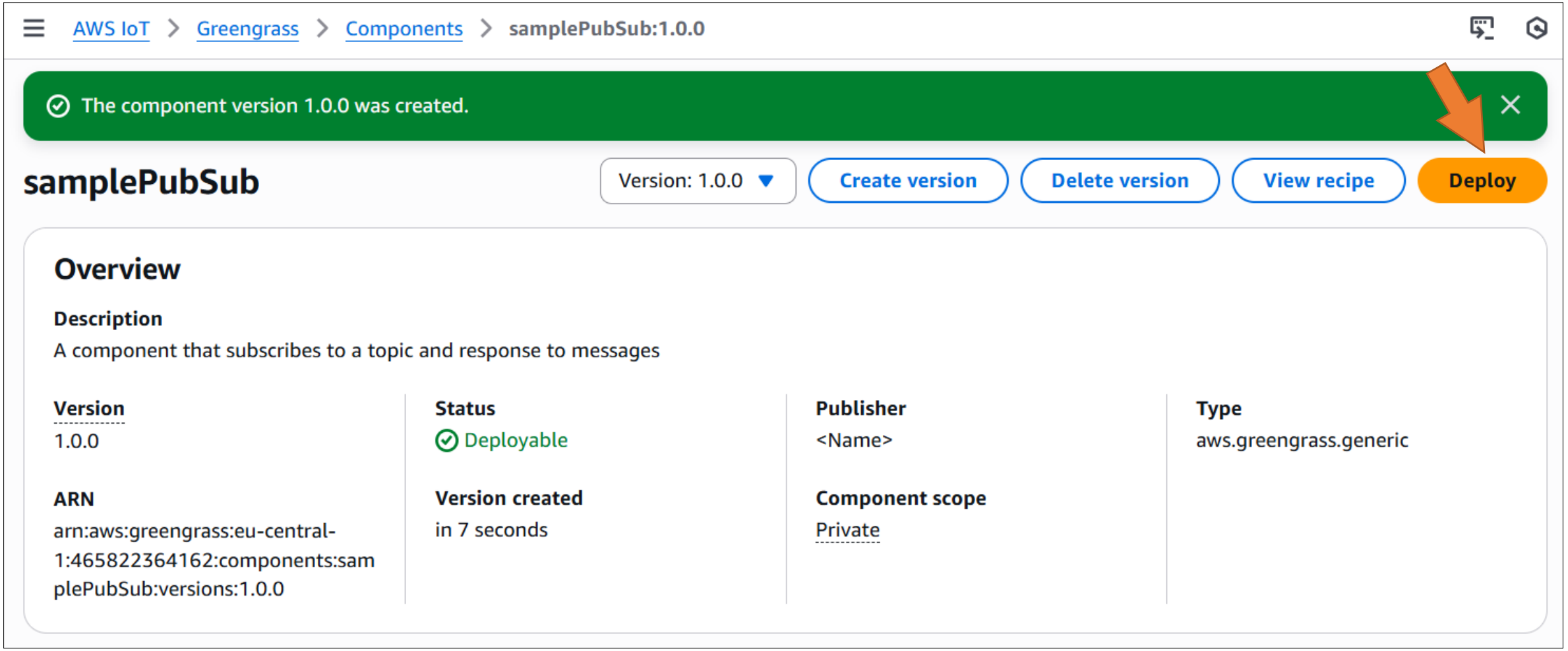
Make sure python3 and pip3 are available on your core device!

awsiotsdk and botocore libraries are required.

★ For simplicity, this demo uses system-level package installation; in production, use an isolated Python virtual environment (venv) instead.

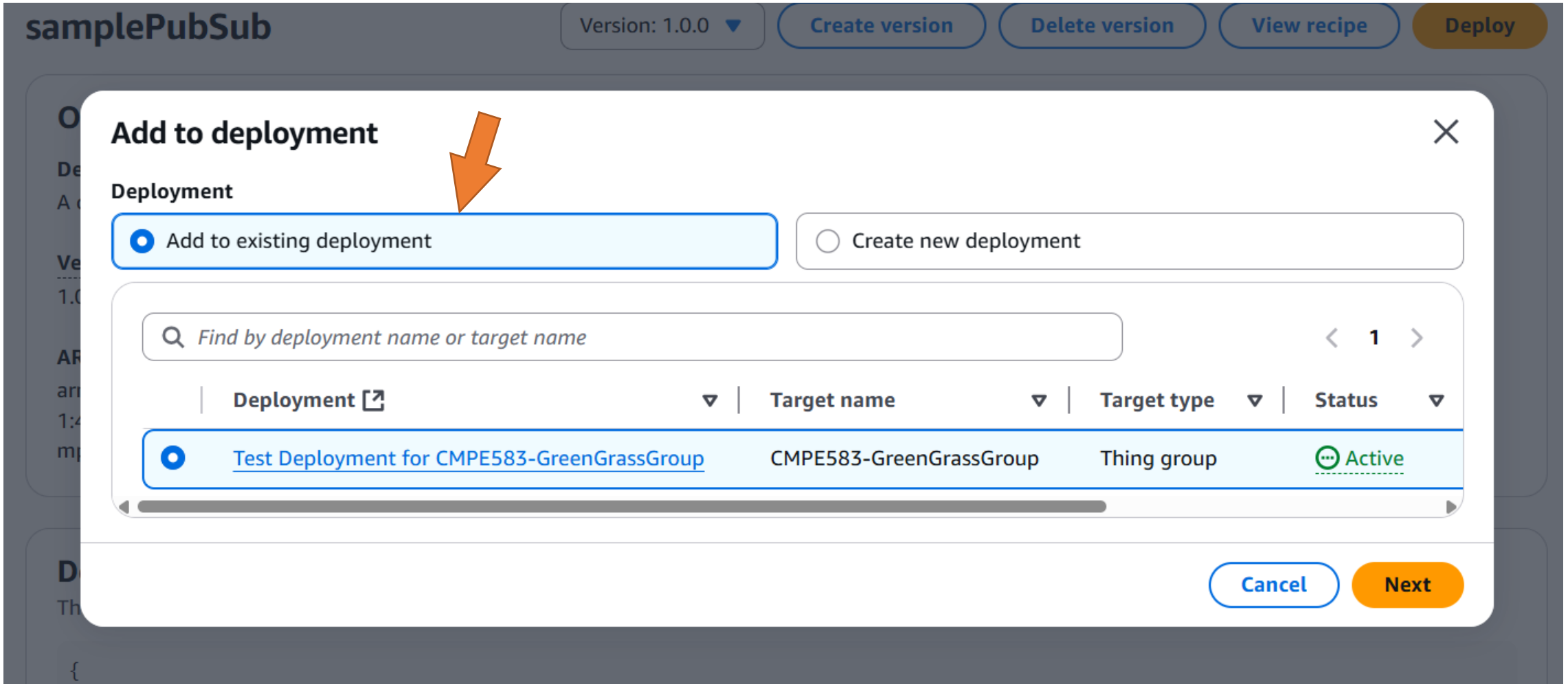
Step 3: Deploy Your Component

Via AWS IoT Greengrass (Console)



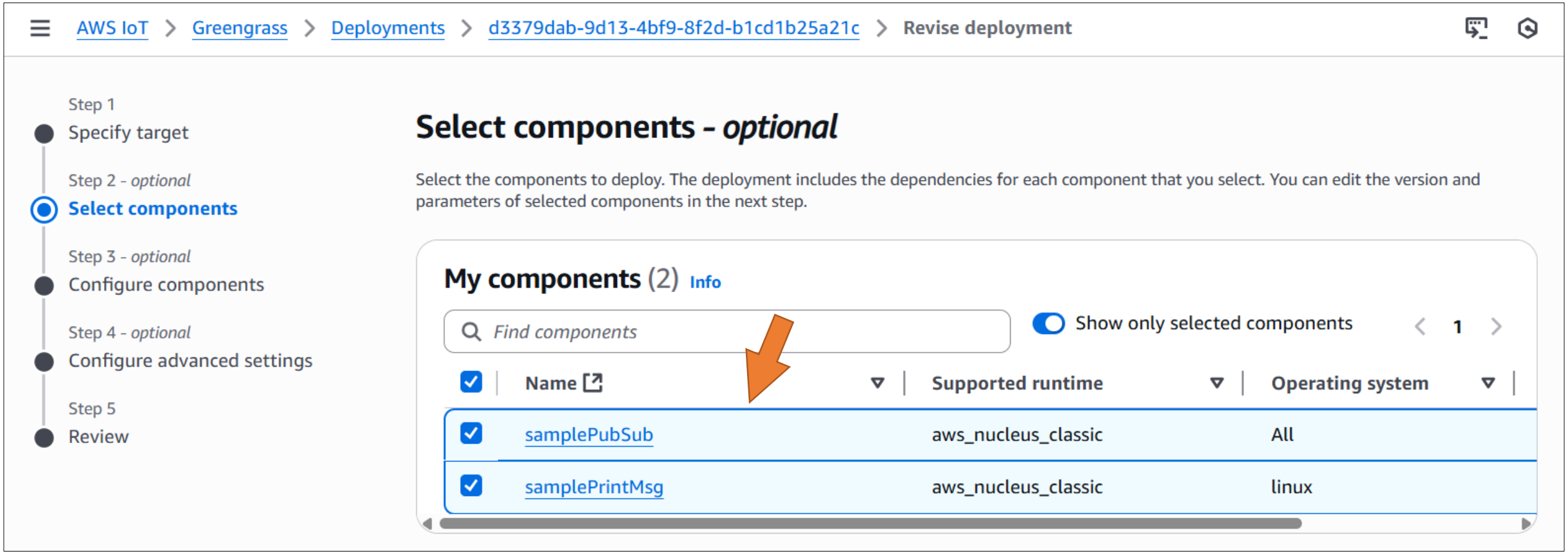
Step 3: Deploy Your Component

Add Custom Component to Existing Deployment



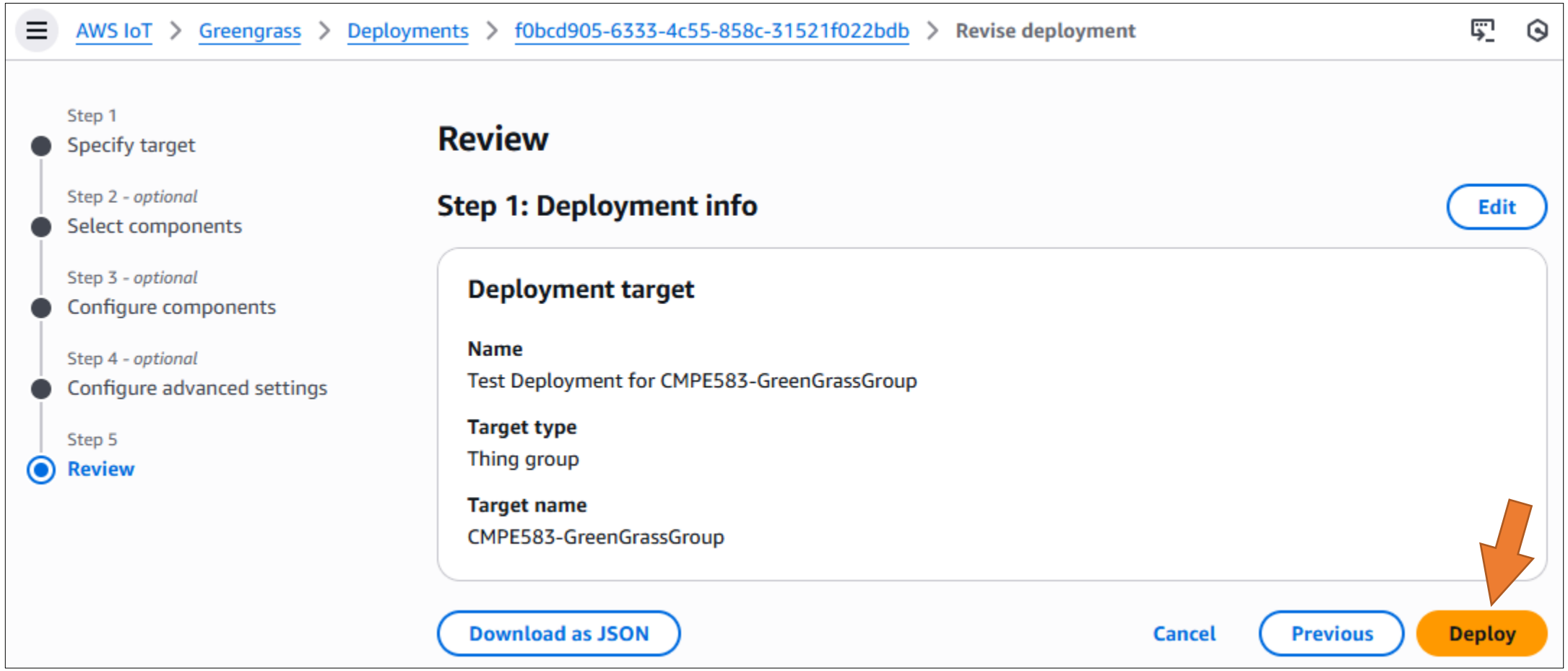
Step 3: Deploy Your Component

Select the Component



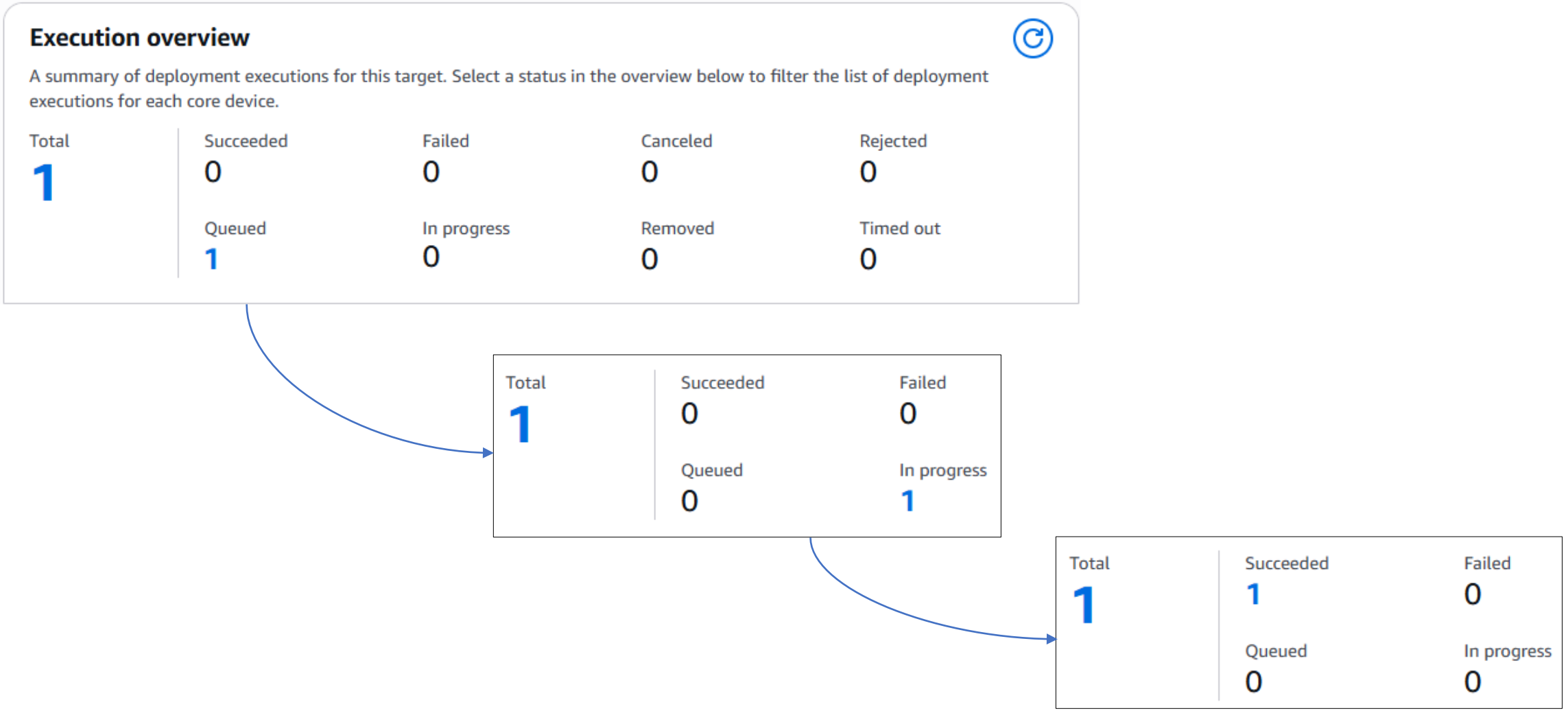
Step 3: Deploy the Component

Review & Deploy



Step 4: Verify the Component

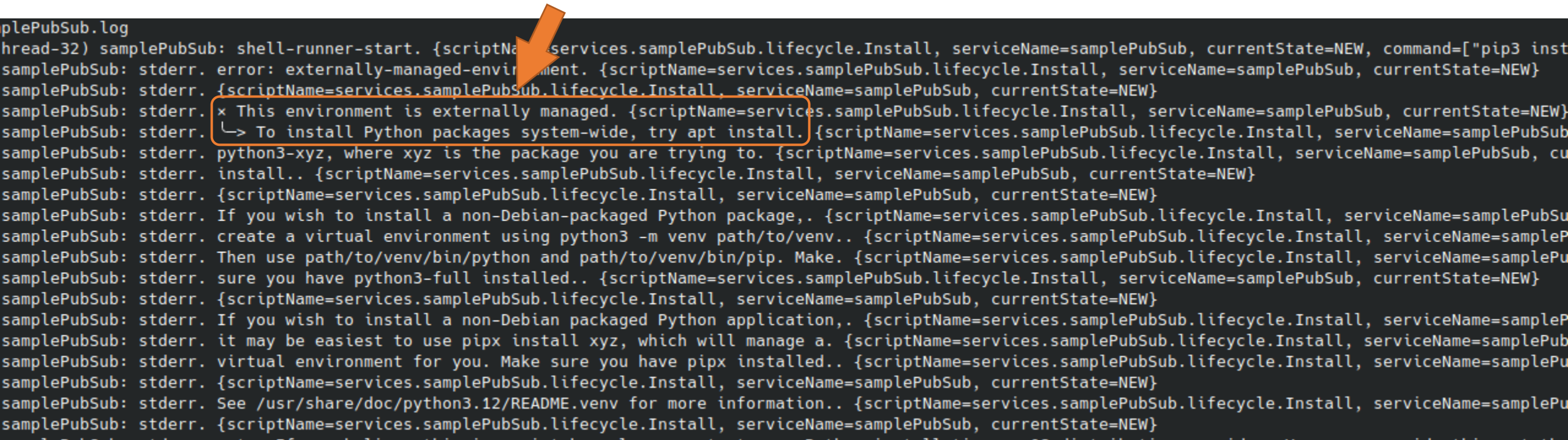
Check AWS IoT Greengrass Console



Troubleshooting

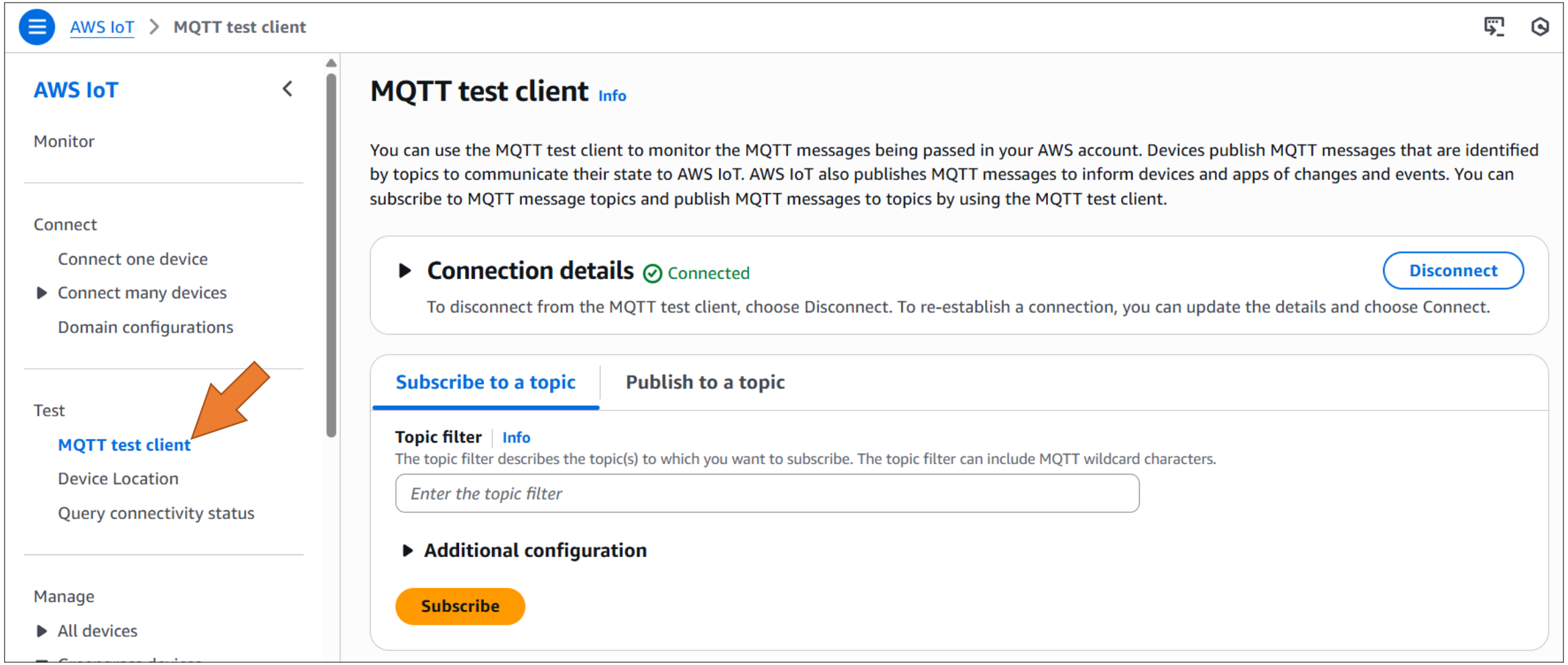
Check greengrass.log File

- Be sure that your Linux environment allows installing Python modules with pip utility!
- Modern Linux distributions mostly prevent installing modules system-wide.
- For this demo we use *pipx* or *--break-system-packages* option with pip3.
- **Recommended**: Use Python virtual environments (venv) to install component dependencies without modifying the system Python environment.



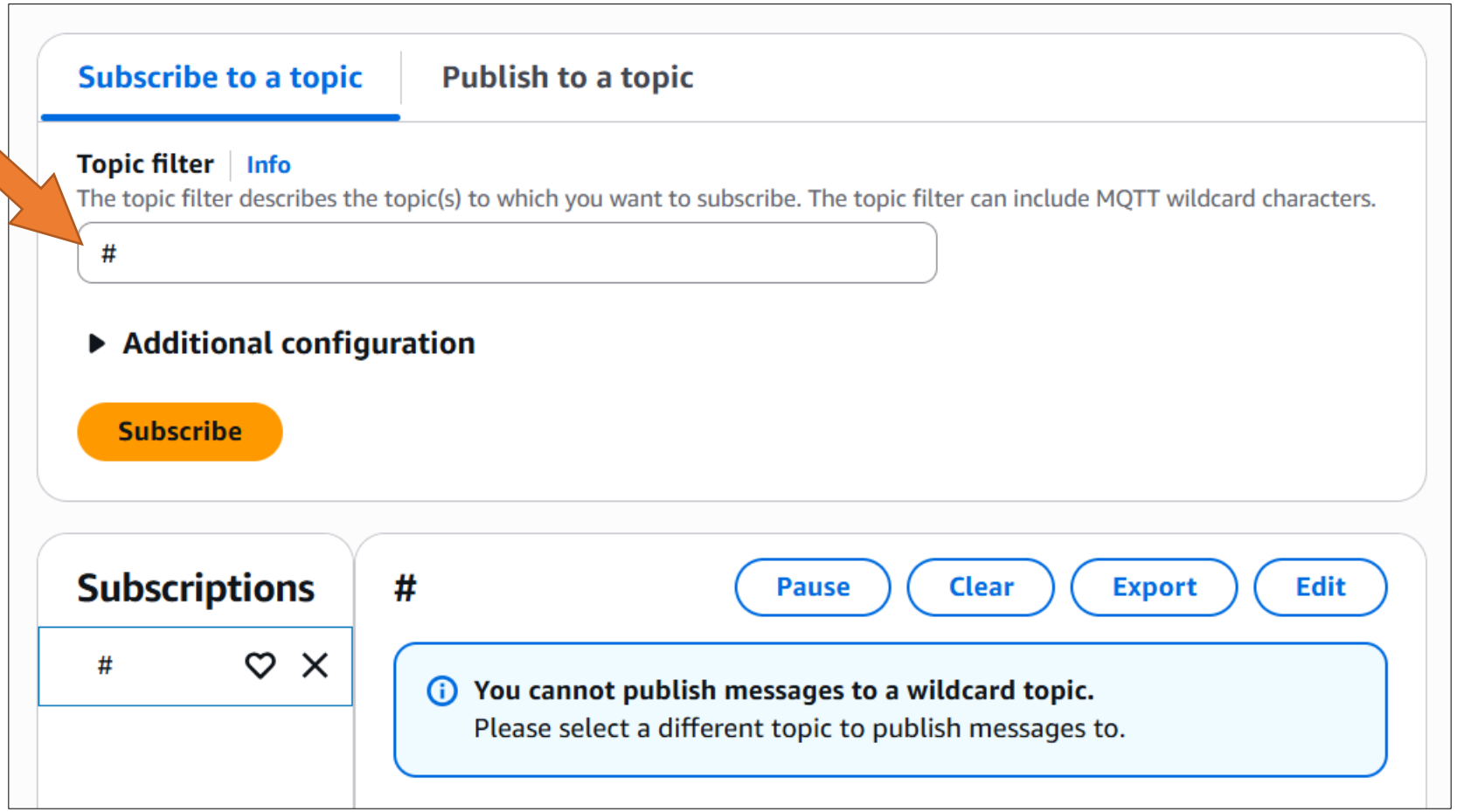
Test Your Deployment

Via AWS MQTT Test Client



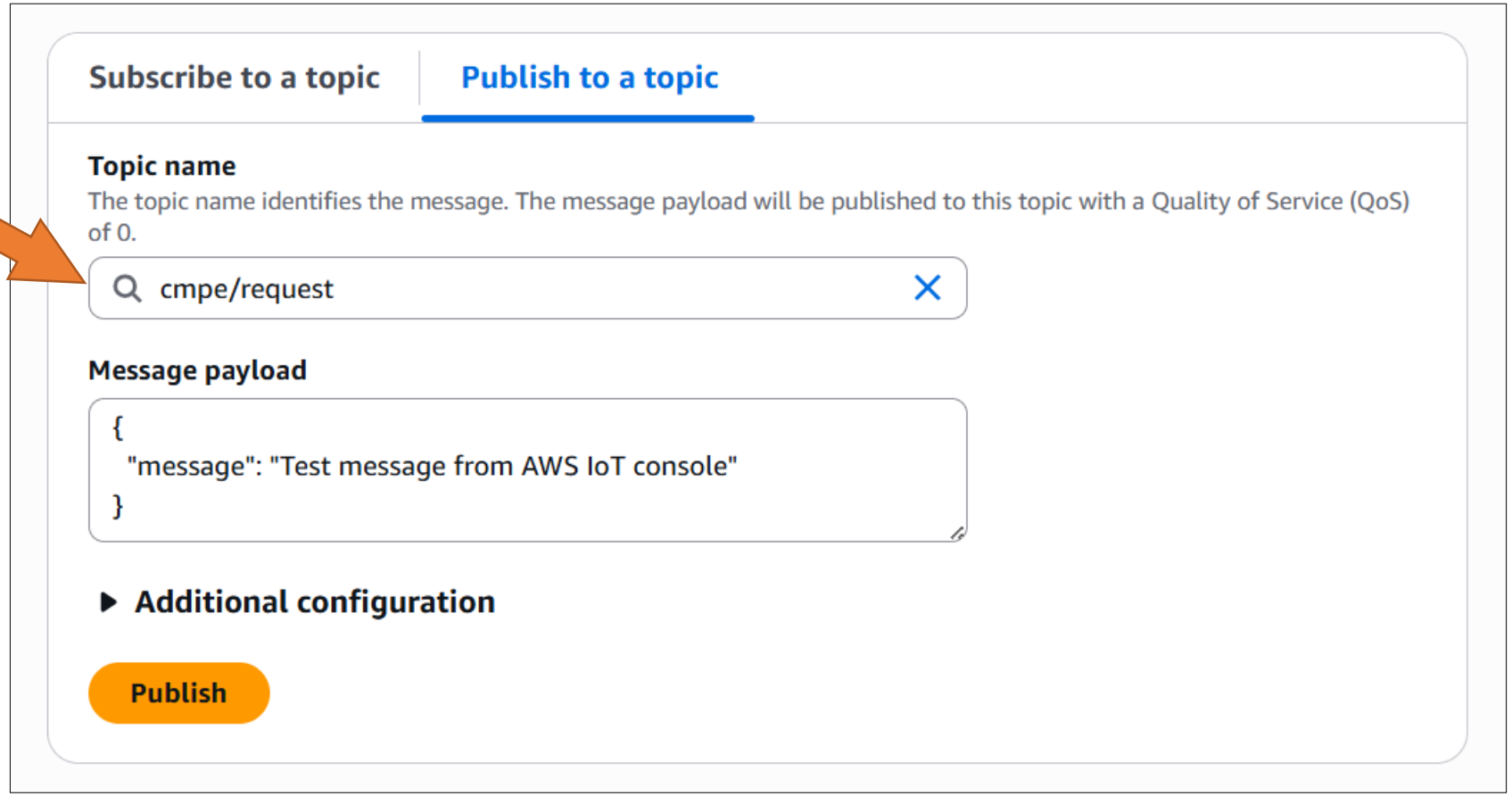
Test Your Deployment

Subscribe All Topics



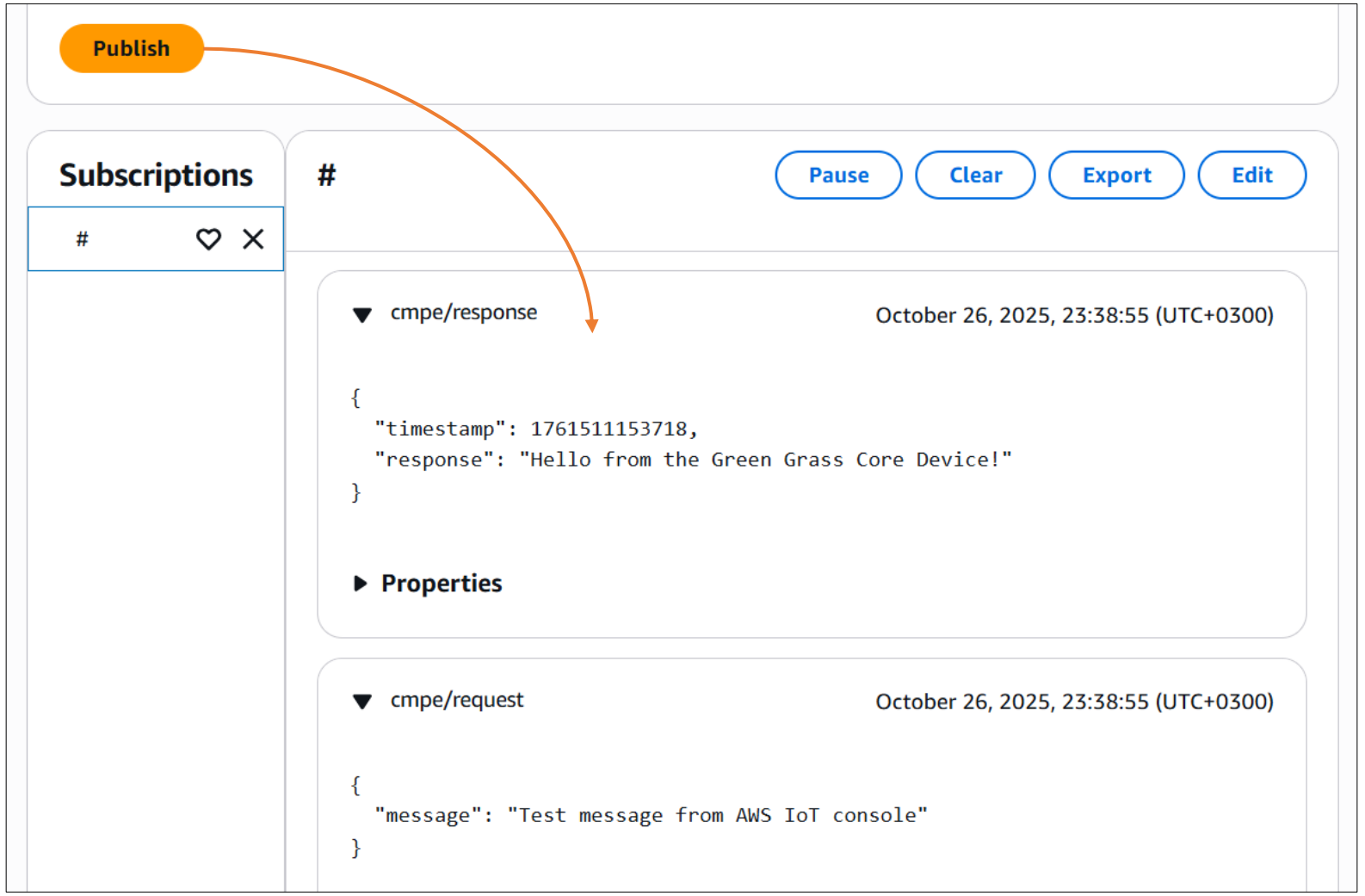
Test Your Deployment

Send a Test Message



Test Your Deployment

Check If the Client Responds Back



What If the Internet Connection is Lost?

CONTINUES LOCALLY

- **Component execution**
  Already-running, offline-capable code
- **Local IPC**
  Publish/subscribe between components
- **Local devices**
  LAN communication when configured
- **Local logging**
  Logs remain available on disk

NEEDS CLOUD CONNECTIVITY

- **Cloud MQTT**
  New AWS IoT Core messages cannot arrive
- **Remote deployments**
  Cloud-initiated updates cannot be fetched
- **AWS API calls**
  S3 and other services are unreachable
- **Cloud telemetry**
  Uploads wait or fail until reconnection

Connectivity resilience Selected outbound MQTT messages can be queued with a configured spooler; retention and delivery depend on settings.

MQTT QoS 0 vs QoS 1

- QoS 0 – “At most once”
  - Message is sent only once.
  - No acknowledgment is expected.
  - Faster, lower overhead.
  - Message may be **lost** if network fails.
- QoS 1 – “At least once”
  - Messages are retransmitted **until** acknowledged (PUBACK).
  - Duplicate deliveries are possible.
  - End-to-end application processing is not guaranteed.

Other Use-Cases of AWS IoT Greengrass Core IPC

- Interact with component lifecycle (component-to-component or component-to-cloud).
  - Pause component, resume component
  - Listen to lifecycle events (start, stop, etc.)
- Interact with component configuration.
  - Get and set component configuration parameters
- Retrieve secret values (access AWS Secrets Manager secrets securely).
  - Gets the value of a secret that you store on the core device
- Authenticate and authorize client devices.
  - Verify the identity of a client device
  - Validates a client device's credentials
  - Verify whether a client device has permission to perform an action on a resource

Thank You